



第4章 数据加密

成都信息工程大学 白杨 副教授

2024年X月X日

第4章

数据加密

本讲内容概要：

01 第一节—密码算法简介

02 第二节—数据存储加密

03 第三节—数据传输加密

01

1.1 密码算法



密码算法简介

密码算法

1.1.1 密码算法简介

在现代社会中，个人的身份信息、银行卡信息、密码等敏感数据的安全性至关重要。这些信息一旦落入不法分子手中，可能导致个人隐私曝光，进而造成财产损失或其他不利后果。

数据加密技术可以有效应对这一挑战。通过将敏感信息转化为密文，即使在数据传输或存储过程中被黑客获取，攻击者也无法轻易解读其含义。

1.1.1 密码算法简介

加密算法主要分为对称加密算法以及非对称加密算法：

对称加密算法：由于我们假定加密算法是众所周知的，所以这就意味着加密密钥 E_k 和解密密钥 D_k 很容易相互推导。因此，如果对 E_k 和 D_k 都保密，则保密性和真实性就都有了保障。但这种体制中 E_k 和 D_k 只要暴露其中一个，另一个也就暴露了。所以对称加密算法必须同时满足保密性和真实性的全部要求，用于加密私人文件。

1.1.1 密码算法简介

非对称加密算法：加密密钥和解密密钥中至少有一个在计算上不可能被另一个导出。因此在变换 E_k 和 D_k 有一个可公开而不影响另一个的保密。

通过保护两个不同的变换分别获得保密性和真实性。保护 D_k 获得保密性，保护 E_k 获得真实性。接收者通过保密自己的解密密钥来保障他接收信息的保密性，但不能保证真实性，因为任何知道他的加密密钥的人都可以将虚假消息发给他。发送者通过保密自己的解密密钥来保障他发送信息的真实性。但任何知道他的加密密钥的人都可以破译消息，保密性不能保证。用于数字签名。

1.1.2 密码算法之AES

高级加密标准（Advanced Encryption Standard, AES）：

算法是最常见的对称加密算法，在该算法中，加解密均是在一个 4×4 的状态（State）矩阵上运作，该矩阵每列4 字节数据也就是 32 比特。AES算法的分组长度固定为128 比特，因此AES的明文列数等于固定值4。

$a_{0,0}$	$a_{0,1}$	$a_{0,2}$	$a_{0,3}$
$a_{1,0}$	$a_{1,1}$	$a_{1,2}$	$a_{1,3}$
$a_{2,0}$	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$
$a_{3,0}$	$a_{3,1}$	$a_{3,2}$	$a_{3,3}$

1.1.2 密码算法之AES

7) AES应用举例

①SSL/TLS协议

SSL (Secure Sockets Layer) 和其后续的TLS (Transport Layer Security) 协议是用于安全地在Internet上进行数据传输的标准协议。AES通常用于SSL/TLS协议中的**加密通信**，确保数据在客户端和服务端之间的安全传输。

②VPN (Virtual Private Network)

VPN是一种安全的网络连接方式，通过加密数据流量，确保在公共网络上的安全通信。许多VPN协议和软件使用**AES来加密传输的数据**，保护用户隐私和数据安全。

③文件加密软件

诸如比特Locker、Veracrypt等文件加密软件通常采用AES算法来加密整个**磁盘或特定文件**，以保护用户的数据安全。

④数据库加密

企业级数据库管理系统 (DBMS) 通常提供数据加密功能，其中包括对数据库中的敏感信息进行AES加密，以确保**数据在存储和传输时的安全性**。

1.1.3 密码算法之RSA

1978年Diffie和Hellman发表了非对称密码的奠基性论文《密码学的新方向》建立了公钥密码的概念。**RSA**是1977年由罗纳德·李维斯特(Ron Rivest)、阿迪·萨莫尔(Adi Shamir)和伦纳德·阿德曼(Leonard Adleman)一起提出的，**其基于质数乘积分解的数论难题**。RSA算法研制的最初理念与目标是努力使互联网安全可靠，旨在解决DES算法秘密密钥的利用公开信道传输分发的难题。而实际结果不但很好地解决了这个难题；还可利用RSA来完成对电文的数字签名以抗对电文的否认与抵赖；同时还可以利用数字签名较容易地发现攻击者对电文的非法篡改，以保护数据信息的完整性。

1.1.2 密码算法之AES

8) AES的优缺点

AES支持不同长度的密钥，包括128比特、192比特和256比特，这代表了其拥有较高的灵活性。

密钥长度的增加通常意味着更高的安全性，但也可能导致更慢的加密速度。

在大规模的系统中，AES算法密钥的分发和管理是保证算法安全性的重要前提，密钥的安全性高度依赖于其存储的安全性。

如果密钥被泄漏，加密数据的安全性就会受到威胁。

1.1.3 密码算法之RSA

3) RSA应用领域

1) 数字证书和公钥基础设施 (PKI)

RSA算法常用于数字证书的生成和签名，用于验证网络通信中的身份和数据完整性。PKI系统使用RSA算法生成和管理密钥对，包括证书颁发机构 (CA) 签发的SSL证书和个人证书。

2) 身份认证和单点登录 (SSO)

一些身份认证和SSO系统使用RSA算法来进行用户身份验证和访问控制，以确保用户身份的安全性和保护系统资源不受未授权访问。

1.1.4 密码算法之MD5

MD5算法是由RSA创始人Rivest设计的，MD5算法是MD4信息-摘要算法的扩展。与MD4相比，MD5牺牲了运算速度但提供了更高的安全性。此算法将对输入的任意长度的信息进行计算，**产生一个128位长度**的“指纹”或“报文摘要”，假定两个不同的文件产生相同的报文摘要或由给定的报文摘要产生原始输入在计算上是行不通的。

1.1.4 密码算法之MD5

2) MD5应用领域

(1) 数据完整性校验

MD5曾经被用于验证文件的完整性，尤其是在下载文件时通过比较文件的MD5哈希值来确保文件未被篡改。虽然不是加密，但在一些简单的数据完整性校验场景下仍然可能使用MD5算法。

(2) 软件校验

MD5曾经被用于校验软件文件的完整性，以确保下载的软件文件未被篡改或损坏。在某些情况下，软件提供方可能会提供软件文件的MD5哈希值，供用户在下载后进行验证。

1.1.5 密码算法之SHA1

SHA1由美国国家安全局设计，是一种**密码散列算法**，并由美国国家标准技术研究所（NIST）发布为联邦数据处理标准（FIPS）。SHA1接受输入消息的最大长度为 $(2^{64}-1)$ 比特，**生成160比特的消息摘要**。

填充消息与被填充消息分组与MD5中处理原始数据的方式类似。

1.1.5 密码算法之SHA1

1) SHA1应用领域

SHA1应用领域举例

(1) 数字证书

一些旧版的数字证书可能使用SHA1算法来生成数字签名。尽管现在推荐使用更安全的哈希算法（如SHA256），但一些旧版的数字证书可能仍在使用SHA1。

(2) **版本控制系统**一些版本控制系统，如Git和Mercurial，曾经在某些情况下使用SHA1算法来生成对象标识符（Object IDs）。尽管现在一般已经转向了更安全的哈希算法（如SHA256），但仍可能存在使用SHA1的情况。

1.1.6 密码算法之SHA2

SHA2是一种密码散列函数算法标准，由美国国家安全局研发，由美国国家标准与技术研究院在2001年发布，属于SHA算法之一，是SHA-1的后继者。其下又可再分为六个不同的算法标准。这些函数是使用来自单向压缩函数的Märkl-Damgaard 结构构建的。它本身是使用专用分组密码的戴维斯-迈耶结构构建的。SHA-2 相对于其前身 SHA-1 进行了重要更改。SHA-2 系列由六个摘要（哈希值）224、256、384 或512 位的哈希函数组成：SHA-224、SHA-256、SHA-384、SHA-512、SHA-512/224、SHA -512/256。SHA-256和SHA-512是新的哈希函数，分别在八个32位字和八个64位字上计算。它们使用不同的移位量和加法常数，但在结构上实际上是相同的，仅轮数不同。对于任意长度的消息，**SHA-2算法都会产生一个256位的哈希值**，称作消息摘要。

1.1.6 密码算法之SHA2

1) SHA2应用领域

(1) 密码学应用

SHA-2算法在许多密码学应用中得到广泛应用，包括密码哈希函数、消息认证码（HMAC）等。

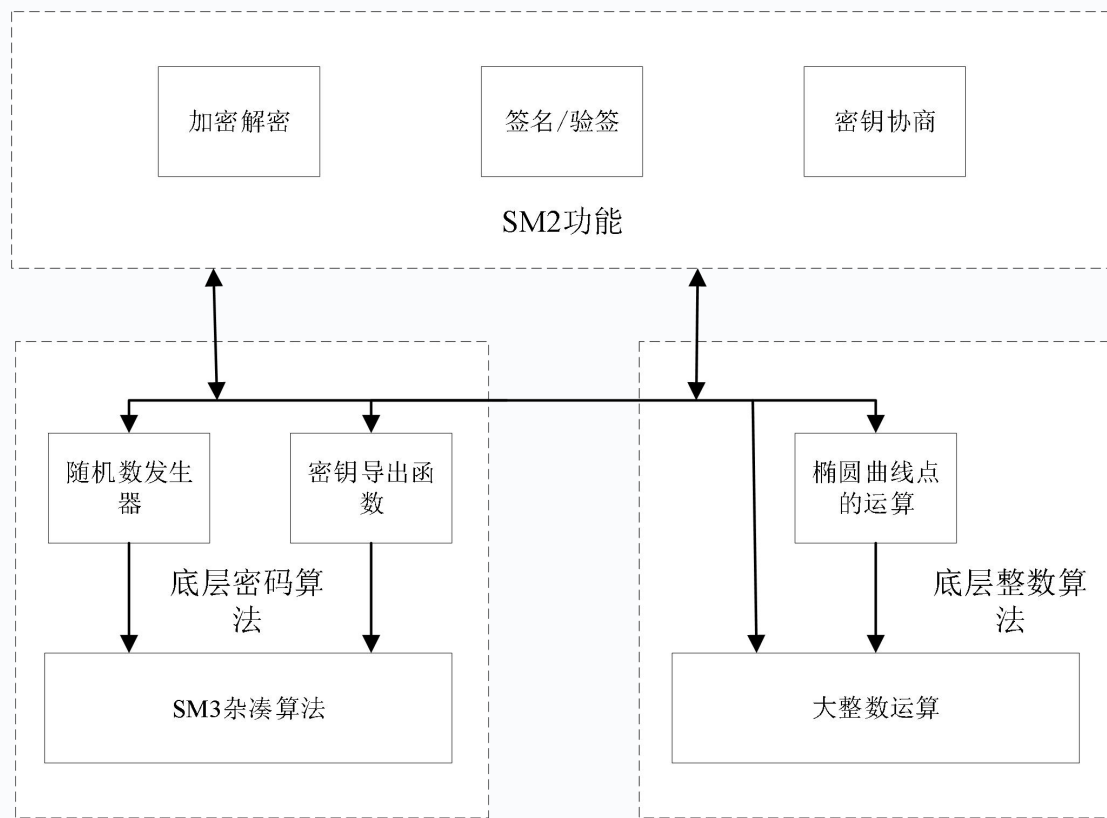
(2) 数字证书

SHA-2算法用于生成数字证书的签名，包括SSL证书、代码签名证书等，以确保数字证书的安全性和可靠性。

1.1.7 密码算法之SM2

我国密码管理部门决定在商用密码体系中，采用SM2椭圆曲线算法替换RSA算法。

SM2是中国密码学算法标准中的一种非对称加密算法。它是由国家密码管理局（中国密码局）于2010年12月17日发布的一种椭圆曲线公钥密码算法。SM2基于椭圆曲线离散对数问题，主要用于数字签名、密钥交换和加密等密码学应用。SM2算法的整体结构如下图：



1.1.7 密码算法之SM2

2) SM2应用领域

(1) 国密算法

SM2作为国家密码管理局推荐的国家密码算法之一，被应用于政府、军队、金融等关键领域的信息安全保护中。

1.1.8 密码算法之SM3

SM3是中国采用的一种密码散列函数标准，前身为SCH4杂凑算法，由国家密码管理局于2010年12月17日发布，相关标准为“GM/T 0004-2012《SM3密码杂凑算法》”。2016年，成为中国国家密码标准（GB/T 32905-2016）。在商用密码体系中，SM3主要用于数字签名及验证、消息认证码生成及验证、随机数生成等方面，可满足多种密码应用的安全需求。SM3是在SHA-256基础上改进实现的一种算法。SM3算法采用Merkle-Damgard结构，消息分组长度为512位，摘要值长度为256位。

SM3和MD5的迭代过程类似，也采用Merkle-Damgard结构。消息分组长度为512位，摘要值长度为256位。

1.1.8 密码算法之SM3

1) 算法概述

输出结果

将得到的A、B、C、D、E、F、G、H八个变量拼接输出，就是SM3算法的输出。

2) SM3应用领域

密码学应用

SM3算法在一些密码学应用中也得到了应用，例如消息认证码（HMAC）等。

1.1.9 密码算法之SM4

SM4（原名SMS4）是中国采用的一种分组密码标准，由国家密码管理局于2012年3月21日发布[1][2]，相关标准为“GM/T 0002-2012《SM4分组密码算法》（原SMS4分组密码算法）”

在商用密码体系中，SM4主要用于数据加密，其算法公开，分组长度与密钥长度均为128比特，加密算法与密钥扩展算法都采用32轮非线性迭代结构，S盒为固定的8比特输入8比特输出。

SM4加密算法属于对称加密算法，2012年3月，国家密码管理局正式公布了包含SM4分组密码算法在内的《祖冲之序列密码算法》等6项密码行业标准。SM4加密解密算法的结构相同，只是使用轮密钥相反，其中解密轮密钥是加密轮密钥的逆序。

1.1.9 密码算法之SM4

1) 算法概述

SM4的轮函数将输入部分看作了4个32位比特长度的数据，每轮的后3个部分都向左移动32比特的数据长度，这三组数据异或后进入非线性部分 τ 和线性部分L，运算后的结果与第一组数据异或置于最后面。如此循环往复32轮，也就是数据一共左移了8个周期，将其中的混乱因素不断扩散至每个比特位中。

2) SM4应用领域

(1) 手机支付应用

在中国，诸如支付宝、微信支付等手机支付应用广泛使用SM4算法来保护用户的支付交易数据的安全，确保支付过程中的数据传输和存储的安全性。

01

1.1 密码算法

1.1.9 密码算法总结概述

算法分类	算法名称	特点	数据安全应用
对称加密算法	DES	分组长度64位	加密通信、磁盘加密、文件加密
	AES	分组长度128位	加密通信、磁盘加密、文件加密
	SM4	分组长度与密钥长度均为128比特	手机支付交易数据安全
非对称加密	RSA	可加密、可签名	数字证书和公钥基础设施（PKI）、身份认证和单点登录（SSO）
	SM2	商用密码体系 基于椭圆曲线离散对数问题	数字签名、密钥交换、加密
散列函数	MD5	输出128位长度	数据完整性校验、软件校验
	SHA系列（SHA1、SHA2、SHA256）	SHA1生成160位输出 SHA-2产生一个256位输出	数字证书、消息摘要、消息认证码
	SM3	摘要值长度为256位	消息认证码

第4章

数据加密

本讲内容概要：

01 第一节—密码算法

➤ 02 第二节—数据存储加密

03 第三节—数据传输加密

数据是企业最宝贵的资源，而存储系统则被视为这些数据的最后防线。随着存储系统从本地连接逐渐向网络化和分布式发展，并与网络上的多台计算机共享，存储系统面临更广泛的攻击威胁。这种变化使得相对静态的存储系统成为攻击者的首选目标，他们试图通过各种手段，包括窃取、篡改或破坏数据，以达到不法目的。在这背景下，确保存储系统的安全性变得尤为关键，成为保障企业信息资产安全的不可或缺的一环。

数据加密是一种有效的手段，用于维护个人隐私的安全性。在现实生活中，我们经常需要存储一些敏感信息，比如个人身份信息、银行卡数据、密码等。这些信息如果被非法获取，可能导致个人隐私的泄露，进而给个人带来经济上的损失或其他不良影响。通过采用数据加密技术，这些敏感信息可以被转化为密文的形式，即使在数据存储的过程中发生信息被窃取的情况，攻击者也无法获得原始信息，从而有效地保护了个人隐私。数据存储加密在提供安全性的同时，为个人提供了一层强有力的防线，防范了潜在的隐私泄露风险。

确保关键数据的安全性、应用系统的功能可用性以及系统可维护性是成功实施数据防护的关键因素。在确定适合响应需求的加密保护技术方案时，我们需要全面考虑这些方面。以下列举了常用的加密技术，并说明了它们应对的安全风险。这有助于制定全面而有效的数据存储加密策略。

数据存储加密分为两类：数据内容加密以及磁盘空间加密

1) 磁盘加密：磁盘采用的块级别加密技术，例如AWS的EBS等，都支持对磁盘进行加密。这种加密的主要优势在于对操作系统是透明的。尽管加密后会导致性能略有下降，但具体下降程度取决于上层应用的不同。AWS的EBS和阿里云的ECS等服务提供了这一加密功能，为企业提供了更安全的磁盘存储选项。

2) 数据库加密：①**透明数据加密（TDE）**是一种数据库提供的加密技术，它实现了对数据文件的实时I/O加密和解密。该技术确保数据在写入磁盘之前被加密，而在从磁盘读入内存时进行解密。重要的是，TDE的应用不会增加数据文件的大小，且无需开发人员对任何应用程序进行修改。密钥管理方面，TDE通过数据库提供的API或组件来实现，使其对开发人员具有透明性。这种方式特别适用于某些场景，例如在磁盘或系统无法对用户开放的条件下（如云环境）。

②数据库加密的另一种方式是通过采用**第三方加固方法**，即将第三方专业数据库加密厂商的产品内嵌在数据库中，以提供透明数据加密功能。所谓透明，即用户应用系统无需进行改造即可使用，而具有权限的用户所见到的仍然是明文数据，使其完全无感知。此外，这种方法还能增强原有数据库的安全能力，例如提供三权分立、脱敏展示等功能。

3) 文件加密：这种加密方式通过在其他文件系统（如Ext2、Ext3、ReiserFS、JFS等）之上堆叠，为应用程序提供了透明、动态、高效和安全的加密功能。通常用于对特定目录进行加密。然而，需要注意的是，这种**加密方式可能导致较大的性能损失**。

1.2.1 数据内容加密

1) 数据库的存储加密是一种关注数据库存储安全的加密方式，旨在防范在存储环节上的数据泄露风险。这一方法主要分为密文存储和存取控制两个层面。密文存储通过采用加密算法、附加密码、加密模块等手段来实现，确保数据在存储过程中以密文形式存储，有效防范了潜在的信息泄露威胁。与此同时，存取控制方面则通过对用户资格和权限进行审查和限制，以防止非法用户访问数据或合法用户越权访问数据。

MySQL中的数据以表的形式存储，表中的每个列都有一个名称和数据类型，用于定义该列中可以存储的数据的种类。表中的每个记录都被称为一行。每一行包含了表中每个列的具体数据值。以利用对称加密算法AES加密MySQL数据库中的字段为例：

MySQL拥有自带加密函数AES_ENCRYPT, AES_DECRYPT, 用于直接加密，我们在添加数据时使用AES算法将信息加密存储：

INSERT INTO User(user_name, user_password) VALUES ('XiaoGang', AES_ENCRYPT('123456','encrypt_key'));

数据经过AES加密后，存储在数据库中的形式为：

	id	username	password
	1	xiaowang	0x4f92c6c47e1e9bb78571d43bdd174f23

1.2.1 数据内容加密

1) 数据库的存储加密是一种关注数据库存储安全的加密方式，旨在防范在存储环节上的数据泄露风险。这一方法主要分为密文存储和存取控制两个层面。密文存储通过采用加密算法、附加密码、加密模块等手段来实现，确保数据在存储过程中以密文形式存储，有效防范了潜在的信息泄露威胁。与此同时，存取控制方面则通过对用户资格和权限进行审查和限制，以防止非法用户访问数据或合法用户越权访问数据。

- **md5**加密
- **ENCRYPT**加密
- **AES_ENCRYPT/AES_DECRYPT**加密解密
- **ENCODE/DECODE**加密解密
- **HEX/UNHEX**结合**AES_ENCRYPT**的加密解密
- **to_base64/to_base64**加密解密

<https://www.cnblogs.com/syw20170419/p/16544618.html>

1.2.1 数据内容加密

那么在查询时，需要使用AES_DECRYPT函数解密：

```
SELECT AES_DECRYPT(user_password, 'encrypt_key') FROM User WHERE  
user_name='XiaoGang';
```

查询结果如下图所示：



	id	username	decrypted_password
☐ 编辑 复制 删除	1	xiaowang	123456

另外除去AES算法，MySQL也支持众多其他算法，例如MD5以及SHA1。

01

1.2 数据存储加密

1.2.1 数据内容加密

2) HTML存储加密

随着互联网的迅猛发展，越来越多的人选择使用网站进行敏感信息的发送、存储、共享和传输。这些信息可能涵盖个人身份、财务状况、医学记录等多个领域，使得网站的安全性变得至关重要。在缺乏安全措施的情况下，黑客有机会窃取或篡改数据，从而导致数据泄露或破坏。因此，HTML网站安全性成为维护网络安全的基础。

网页防篡改技术已成为政府、金融、电商等领域的重要安全保障手段。政府机构的官方网站利用此技术，保障发布信息的真实可信；金融机构的网上银行系统采用网页防篡改技术，确保用户资金安全；而电商平台则依靠此技术防止商品信息被篡改，维护市场秩序。这些领域广泛应用网页防篡改技术，不仅增强了网络安全，也提升了用户的信任感和交易体验。

网页防篡改技术	具体内容
事件触发技术	采用程序对网站目录进行实时监控，一旦发现异常情况，立即触发检查机制，以确定是否存在非法篡改行为。
核心内嵌技术	在用户请求访问网页时，在系统正式提交网页内容给用户之前进行完整性检查的技术。通过在网页内容中嵌入独特的标识符或指纹，系统能够在传输过程中验证网页的真实性和完整性，防止篡改行为的发生。
定时循环扫描技术	使用程序按用户设定的时间间隔定期扫描网站目录的方法。一旦发现网站内容被篡改，系统将立即使用备份数据进行恢复，以确保网站数据的完整性和安全性。
文件过滤驱动技术	利用系统底层文件过滤驱动技术，拦截并分析IRP流。通过监控和分析系统底层的文件操作流，系统能够及时捕获和阻止任何对网站文件的非法篡改行为。

1.2.1 数据内容加密

下面我们用购物网站的SQL注入攻击来举例，用户可通过搜索框查找自己想要的商品，比如：袜子，如果系统是利用SQL语句进行查询，那么用户的请求在后台即为：

```
SELECT * FROM products WHERE product_name = '袜子';
```

这样系统处理查询语句，再通过网页返回袜子等商品，

但是攻击者如果试图通过恶意注入SQL代码来绕过正常的搜索逻辑，尝试访问数据库中的未授权数据或执行恶意操作，如下：

```
SELECT * FROM products WHERE product_name = " OR '1'='1'; --";
```

这个查询条件始终为真（'1'='1' 是恒真的条件），导致系统返回所有商品，而不仅仅是包含手机的商品。这便对返回结果进行了篡改，并且在这个基础上，还能做出很多危害信息安全的攻击，比如说查看所有用户名及其密码。但在这里，也涉及到了数据存储安全，由于在前面章节中已经讲解了数据内容加密，故在此不再描述。

网页防篡改技术主要关注网页内容的完整性和安全性，而HTML源代码加密技术则着眼于保护HTML代码的机密性和防止代码泄露，使Html代码不会轻易被他人获取。

JShaman的Html源代码加密：<https://www.jshaman.com/enhtml/>

输入你需要加密的Html源代码，然后点击加密

1.2.1 数据内容加密

加密选项中包含JS混淆加密，僵尸元素植入，链接加密，SEO优化。

JS混淆加密：对Html编码后的JavaScript代码进行混淆加密，使代码无法阅读、理解。

并且是混淆加密是多态特性，一次一结果，永不重复。

僵尸元素植入：给页面中随机插入div、span、p等元素，形成新的节点，这些节点中包含内容，但不显示、不影响页面布局。由于其真实存在，因此会对DevTool（浏览器开发者工具）造成干扰，使用无法从“DOM和样式探查器”中直接复制页面内容。

链接加密：对网页中所有链接（“a href”语句）进行加密，隐藏链接地址。以此防止链接被获取、防止爬虫根据链接获得其它页面地址。

SEO优化：使加密后的Html代码中包换与原页面相同的title、keywords、description，及全页面渲染后展示的文字内容。以此增加页面对搜索引擎的友好性，使网页更容易被收录，并有排名优化效果。

加密完成之后，我们就得到了加密的HTML的源代码

01

1.2 数据存储加密

1.2.2 磁盘空间加密

在数据内容加密技术和磁盘空间加密技术之间，存在着不同的加密重点和应用场景。数据内容加密技术通常用于保护特定文件、文件夹或数据块的安全性，通过加密这些数据内容，确保即使在存储或传输过程中被未经授权的访问者获取，也无法轻易解密或篡改其中的信息。这种加密技术常见于涉及敏感数据的场景，如个人隐私文件、机密文档等，以满足对数据保密性的高要求。

以下是常见的磁盘加密技术

加密技术	具体内容
比特Locker硬盘加密	比特Locker是由微软开发的一种全盘加密技术，专为Windows操作系统设计。它通过对整个磁盘的数据进行加密，包括操作系统、应用程序和文件，以确保对磁盘内容的全面保护。比特Locker采用高强度的加密算法，例如AES和XTS模式，以确保数据的机密性和完整性。
FileVault硬盘加密	FileVault是由苹果公司专为Mac OS X开发的硬盘加密方案。它采用XTS-AES-128加密算法，对整个硬盘进行加密保护。用户只需设置一个密钥，在计算机启动时即可解锁硬盘。通过FileVault加密，用户的数据得到了有效的保护，确保未经授权的人无法访问加密的数据。
VeraCrypt硬盘加密	VeraCrypt是一款开源的硬盘加密工具，支持多种操作系统，包括Windows、Mac和Linux。该工具采用强大的加密算法，如AES、Twofish和Serpent，可以对整个硬盘或指定的文件夹进行加密。VeraCrypt提供了多种加密模式和密码强度选项，用户可以根据自己的需求进行灵活的设置。
TrueCrypt硬盘加密	TrueCrypt是一款免费的硬盘加密软件，支持多个操作系统，包括Windows、Mac和Linux。该软件能够对整个硬盘或指定的分区进行加密，采用多种强大的加密算法，如AES、Twofish和Serpent。TrueCrypt提供了多种加密模式，例如系统加密、隐藏容器等，使用户能够根据具体需求选择适用的加密方式。

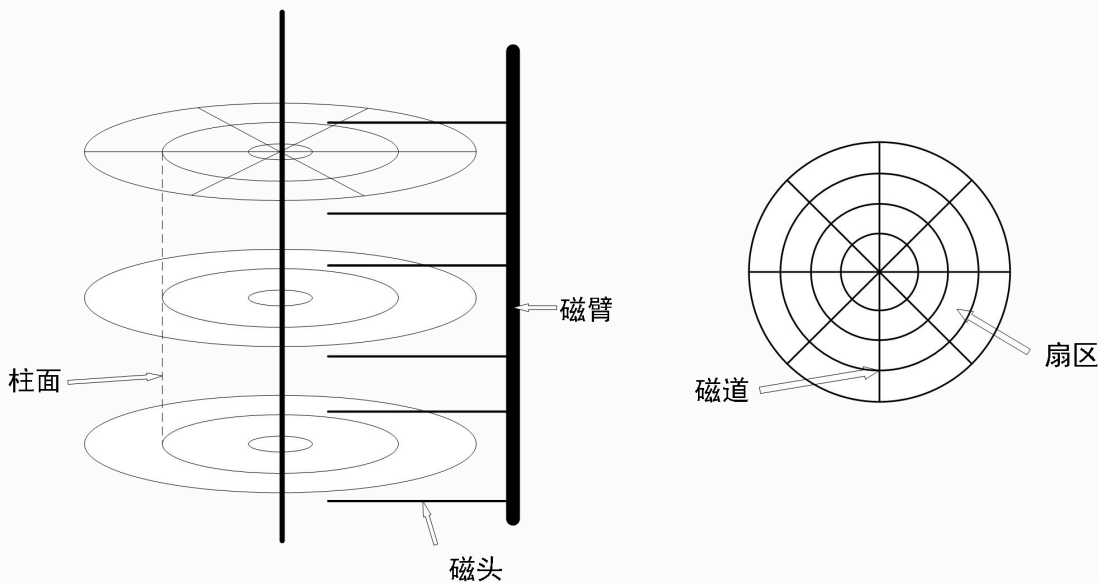
1.2.2 磁盘空间加密

磁盘的物理结构是比较复杂的，如右图所示，这里我们只需要知道最常用到的几个术语即可：

扇区 (Sector)：将磁道划分为若干个小的区段，就是扇区，每个扇区的容量为512字节。

磁头 (Heads)：每张磁片的正反两面都各有一个磁头，一个磁头对应一张磁片的一个面，因此我们能知道用第几磁头就表示数据在哪个磁面。

柱面 (Cylinder)：所有磁片中半径相同的同心磁道构成“柱面”，柱面数等于磁道数。



1.2.2 磁盘空间加密

我们将详细说明比特Locker硬盘加密的原理：

比特Locker是Windows操作系统中的磁盘加密工具，用于提供数据的加密保护。在比特Locker的运行过程中，它支持两种主要的工作模式：透明运行模式和用户认证模式。

(1) 比特Locker的透明运行模式是一种无需用户干预的操作模式。在这种模式下，比特Locker使用TPM (Trusted Platform Module) 硬件来检测计算机的启动环境是否有未经授权的改变。TPM是一个安全芯片，用于存储密钥和执行加密操作。比特Locker通过TPM来确保启动环境的完整性。如果检测到任何未经授权的更改，如BIOS或MBR的修改，比特Locker会触发安全措施。

(2) 比特Locker的用户认证模式中，除了使用TPM外，还需要用户的额外身份验证。这通常涉及到用户输入PIN码、密码或使用外部USB设备（如智能卡）进行身份验证。当启动环境检测到未经授权的更改时，比特Locker会要求用户提供恢复密钥。用户必须插入预先配置的USB设备，该设备包含恢复密钥。恢复密钥用于解密卷主密钥，使系统能够继续引导过程。

比特Locker的加密原理：比特Locker首先与操作系统进行交互，进行初始化设置，包括加密密钥的存储位置和磁盘大小等。然后，比特Locker对磁盘驱动器中的每一个扇区进行加密。扇区加密的过程涉及使用密钥对数据进行加密和解密，以确保在传输和存储过程中的数据安全性。

1.2.2 磁盘空间加密

比特Locker在操作系统中实施了访问控制机制，只有拥有正确密钥的用户或程序才能访问加密的扇区。这种访问机制有效防止了未经授权的访问和数据泄露。此外，比特Locker采用一个密钥来加密磁盘驱动器中的数据，通常将该密钥存储在受保护的位置，如硬盘设备或云存储服务中。密钥的管理和分发受到严格的权限和控制，以确保数据的安全性。

第1章

数据加密

本讲内容概要：

01 第一节—密码算法

02 第二节—数据存储加密

➤ 03 第三节—数据传输加密

数据传输加密的存在是为了应对数据传输过程中的安全威胁和风险。在网络通信中，传输的数据往往包含敏感信息，如个人身份信息、银行账号、密码等。如果这些数据在传输过程中以明文形式存在，就容易被窃取、篡改或伪造，造成严重的信息泄露和安全问题。因此，采用数据传输加密技术对数据进行加密处理，能有效保护数据的机密性和完整性，确保传输过程的安全性。

在数据传输加密中，常用的加密算法包括对称加密算法和非对称加密算法。对称加密算法如AES（高级加密标准）和DES（数据加密标准）能够快速加密大量数据，并提供较高的安全性；而非对称加密算法如RSA（RSA算法）则能够实现更安全的数据传输，通过公钥和私钥实现加密和解密，保障了数据传输的可信任性。同时，数据传输加密还可以结合身份鉴别技术，校验传输节点的身份，防止传输节点被伪造，提高了数据传输的安全性和可靠性。

数据传输加密分为：信源加密和传输通道加密

1.3.1 传输信源加密

信源加密 (Source Encryption) 是一种信息安全技术，其核心思想是在信息的产生源头对数据进行加密，以确保数据在传输、存储和处理过程中的机密性和完整性。通过在信源加密阶段保护数据，可以有效防止未经授权的访问或篡改，即使数据在传输或存储时被截获，也无法被非法解读。

信源加密的主要特点

- 1. 加密位置：**数据在产生源头（例如传感器、设备或系统的采集端）被立即加密，而不是等到传输或存储阶段才进行保护。
- 2. 端到端安全性：**配合终端解密机制，可以实现数据从生成到消费的全生命周期保护。
- 3. 实时性强：**由于加密在数据产生时进行，适合实时性要求高的应用场景，如物联网设备、实时视频流、或金融交易系统。
- 4. 减少攻击面：**通过在信源加密，可以降低数据在传输路径中被劫持或窃取的风险。

1.3.1 传输信源加密

信源加密的实现机制

1. 对称加密：

- 例如AES算法，可以在信源快速加密数据。
- 优点是加密速度快，适合高频数据流。
- 缺点是密钥管理复杂，需要确保密钥在传输端和接收端的安全性。

2. 非对称加密：

- 使用公钥加密（如RSA）对数据加密，接收端使用私钥解密。
- 优点是安全性高，适合敏感信息的保护。
- 缺点是加密速度较慢，难以处理高频率数据。

3. 哈希与签名：

- 通过结合哈希算法和数字签名验证数据的完整性和来源的可信性。
- 适合需要附加认证的场景。

1.3.1 传输信源加密

信源加密的应用场景

1. 物联网（IoT）：传感器或智能设备在采集数据后立即加密，保护数据免受中间人攻击。
2. 视频监控：实时视频流在摄像头端加密，确保敏感画面不被非法访问。
3. 工业互联网：工业控制系统中的关键数据在生成时加密，以防止生产流程被恶意干扰。
4. 医疗数据：医疗设备采集的患者隐私数据在信源加密后传输到服务器，确保隐私合规。

信源加密的意义

- 增强数据安全性：数据的保护从源头开始，可减少因传输或存储环节引发的泄露。
- 提高信任度：通过加密保障数据真实性，尤其在需要数据共享的场景中显得尤为重要。
- 符合法规要求：如《个人信息保护法》和《数据安全法》中对数据安全保护的高标准。

在实践中，信源加密不仅是技术手段，还需要结合密钥管理、安全协议及访问控制策略，才能真正实现数据安全的全流程保护。

1.3.1 传输信源加密

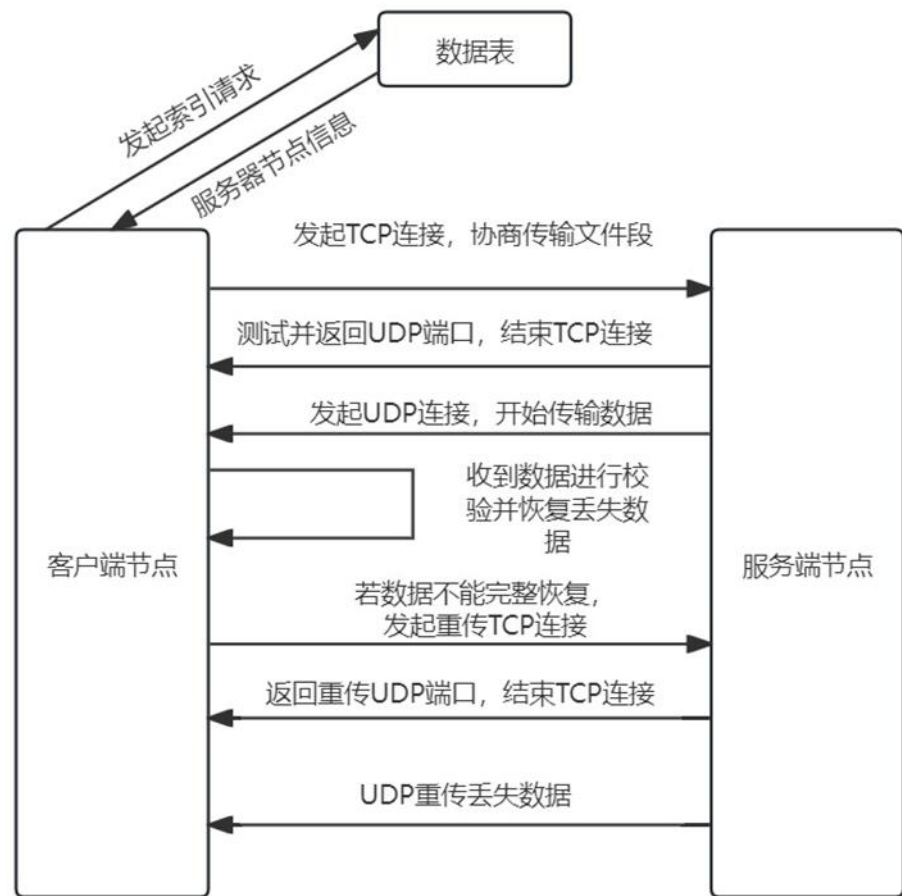
保护数据安全已经成为国家安全的核心任务之一。我们必须重点关注防止关键信息基础设施、核心数据、重要数据以及大量个人信息受到外国政府的影响、控制或恶意利用的风险。此外，我们还必须警惕供应链中断对能源、交通、水利、国防科技等国家关键信息基础设施连续性造成的危害。

数据加密技术与防火墙配合使用，是提高信息系统和数据安全性的重要手段之一。它能有效防止秘密数据被外部破解，保护数据的机密性和完整性。在当前情况下，数据信源加密仍然是计算机系统中最可靠的方法之一，用以保护敏感信息不受未经授权的访问。传输信源加密的核心是密码学。信源加密利用密码技术对信息进行加密，实现信息隐蔽，从而起到保护信息的安全的作用。

1.3.1 传输信源加密

1) P2P与TLS

实现点对点加密技术主要采用了两种协议：Peer-to-Peer(P2P)协议和Transport Layer Security(TLS)协议。P2P协议是一种端到端加密的通信协议，它允许两个设备之间直接通信，而不需要通过中心节点进行连接。在P2P协议中，每个设备都充当了服务器和客户端的角色，能够直接相互通信。如右图为P2P协议的工作方式：



1.3.1 传输信源加密

而TLS协议是一种加密协议，它在传输层上对数据进行加密。TLS协议采用了非对称密钥加密方式来保证数据的安全性。

当使用P2P协议和TLS协议结合时，就可以实现点对点加密通信。在这种方式下，每个设备都会生成一对公私密钥。当设备需要与另一个设备建立安全通信时，它会将自己的公钥发送给对方设备，对方设备在收到公钥后，使用该公钥对通信内容进行加密，然后将加密后的数据返回给发送方。发送方使用自己的私钥对数据进行解密，以获取原始数据。这样通过P2P协议连接和TLS协议加密，两个设备之间就可以实现安全通信了。

下面我们来观察P2P与P2P+TLS两种协议之间的区别：

假如用户与服务器利用P2P进行通信，客户端向服务发送123、456、789三次数据，服务器回复“消息已收到”：

```
请输入要发送的消息： 123
收到回复： 消息已收到
请输入要发送的消息： 456
收到回复： 消息已收到
请输入要发送的消息： 789
收到回复： 消息已收到
```

1.3.1 传输信源加密

我们将传输过程中的信息进行抓取：

123消息已收到456消息已收到789消息已收到

我们可以看到消息是以明文方式传输的，一旦攻击者窃取到信息，便可以知晓所有的数据。那么P2P+TLS会有什么不同呢？同样是客户端向服务发送123、456、789三次数据，服务器回复“消息已收到”：

```
连接成功！
请输入要发送的消息：123
收到回复：消息已收到
请输入要发送的消息：456
收到回复：消息已收到
请输入要发送的消息：789
收到回复：消息已收到
请输入要发送的消息：|
```


1.3.1 传输信源加密

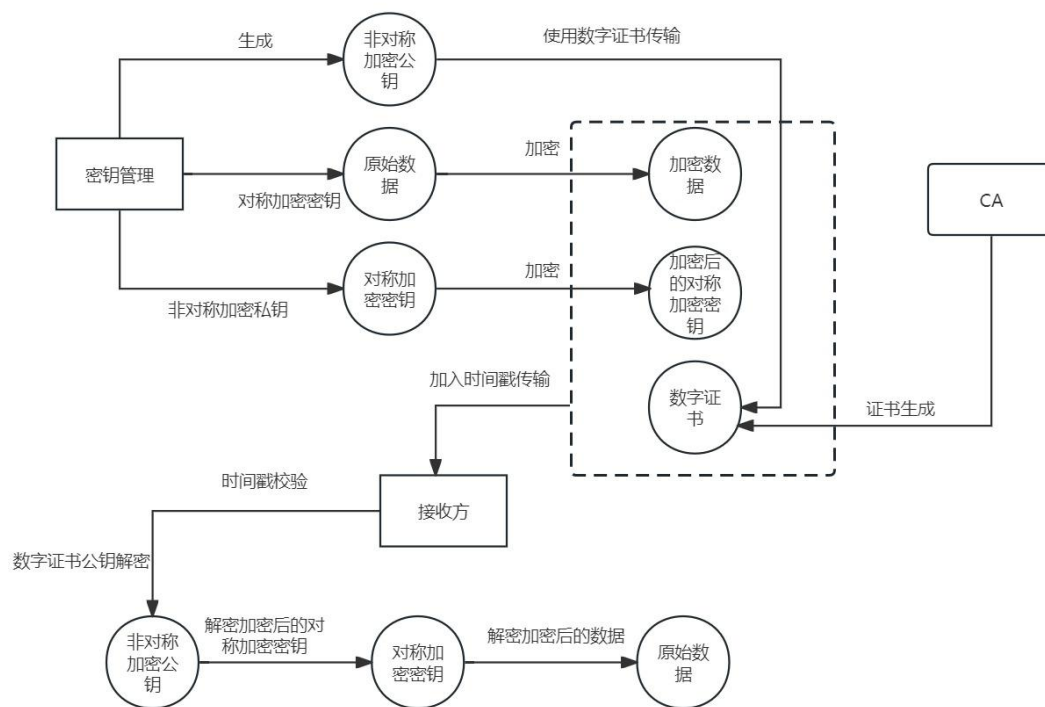
我们将传输过程中的信息进行抓取：

```
ie7W/lcNK9ZFIL5fmEVoieGEEOLPCbMhVoxVkBvHeIeUgT299iPlDv2JU5MmWSBAYs6aRTKcH4ArVRFuL3Ty6rNl5iz8sS3W
DnlNQdj3MUyx8bBMFu834S2MxIcnCdS/d/xdgDlWynJThDPHJiT8eUaCiXNYJR5RwevhKWtfSkB6Eu/sRWSthQj7eee+h+vu
XWUFW2XQM8R01aJonsnkb7dYwgdphI3ewgBH827bYqhg5MOMr9zqp01qOxRaSLgdA8QnxXwxIyejNpgtFJt+m3ltQfeEkUAF
PFF1QC0eVzaJtHzPoB0yL7PZ2im8rJBysvO96++LmiV9jqo07/awbha9Sf2gmklphZj/Pj6qYTJjaPyVcFSYQwjy6ehGQy9Mz
NFZjn1wR+sGOVqmbpnVbCqdnOCI7E5NruwzSLZo8lhu3t0pwpe6mq2Dj2kwU73UZcX67U0Qf9uTxVRF24UIfkZIEfwCP5wGa
BPUOPQsLHaQfq7D9YSkwWiQ7fVTz4kEm消息已收到RlbFljjKdEUyzZfVkkZRw/CXRc1X73p3jRzlgL7YF4VSKk9v9ByHnJ7
VhjyFs0HgZbYbHQRYTNlqx5hE08iVo1RWGklk1vG3IzfVrKlnbS68cVnYYXiZHPz9WE1UGeWxpDxqqXkT6/jQ000QUkNOrYJ
xsoEjnSNIFGrL1La9ImmRbOL9KKzI+aJDK8wdjJEGve7859/nY5eYr/ic5OK6oFoXnFHDcwRzSXAtsEyJSbYuXwt3jTCyo4H
n1Euj5zrStDNttcDl+C/XuGb49G/dS1l7K5KUThAFnYEpawNSvgAnbzwY88zAv8dWMCZjrZH3tXnjUzulUB2iS9jNxiG5k8/C
hR7LpWrkV4Dl/QK/tF1uPnoA9Zcne5VSYPEbd3y1h6xC1gORrY4Ey44C1lKDNJFIy3CAKMeDwZyII8XhGKVqqYZOGzvHsE08
5794QHkbZ0xnk9iMq4ctyLoiVzKz45DHUZL7TN4mZQp1EbGdS0CYhySqNjNWEIi4J/n0JeTg7消息已收到dYDlKLpg4LfbyV
G9K0uwqEmudl0/ty7SDNzt4/ExCDGAoXxfpQkmFJqTCvIwRhsP3wQWiatpcql7o5sUu0Cn9JDLOxkb+FBsT0R03Vsb18UOK0
X+BxTrhHPAsNjcI82Wv9ks20W+bFrPZvihiz0OCJf1Mae+v3Dy+d89/5o4II35366j7D2WQHwszoQQ0Z36ilM3uYYIUrYbOB
balNcNLI88ufUR++gitgun23Dkg/5ozNWbpQbZH++khUeDgnaxhXEj6frrafxusUuwI1MMxET/sp4e/F8mV84/NRmTvRhGlg
/zjfrRu0/UyxkTLtkI5mlteFw6TRLoKvaYlGyBtvBeNhvmSoVMCA1bdzxGmHfnRr18rdUQ6iF8PZgbB7x0fVoeuZJPd7hBXN8
rIUNysdgts4cTrBzZXJ/y5cB5GuFvAUhck1koIGPGDS4vLvt01mXL0Ijrj+lmZqQW5hGkRNubF9pOoh+BmeroStXftwEsx1n
jk00v/ImZPv+bAFU6N消息已收到
```

我们可以看到，客户端使用服务器提供的公钥对消息进行加密后再进行传输，而服务器则使用自己的私钥进行解密，即使攻击者在传输过程中抓取到了相应的数据包，也需要通过破解密码算法才能够获得准确的消息。

1.3.1 传输信源加密

下图为数据在传输过程中保障机密性的流程：

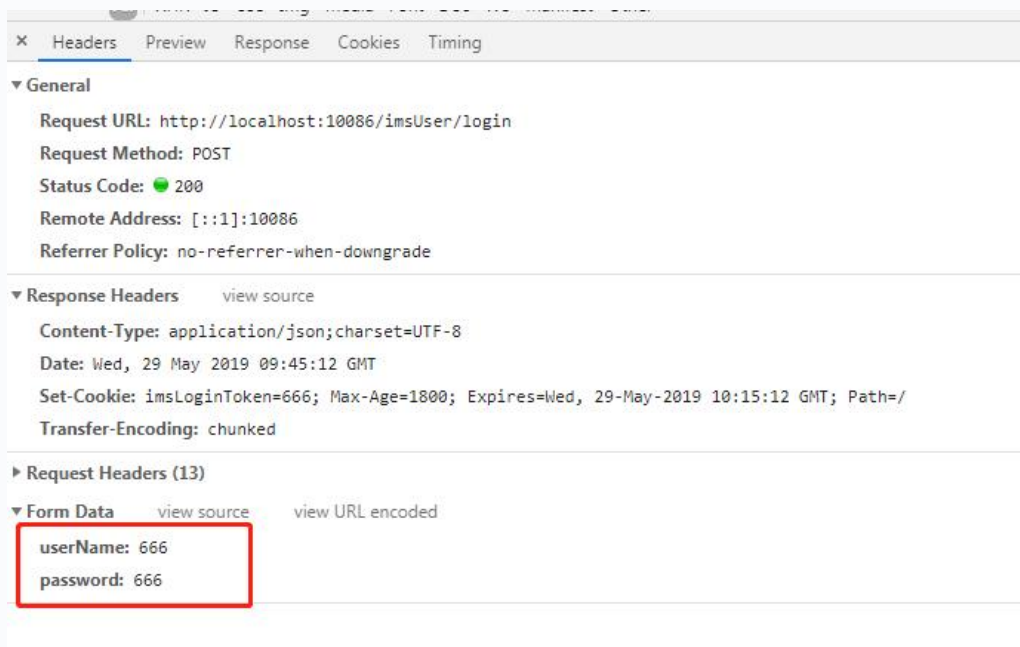


1.3.1 传输信源加密

一些在数据传输过程中保证机密性和完整性的技术如下：

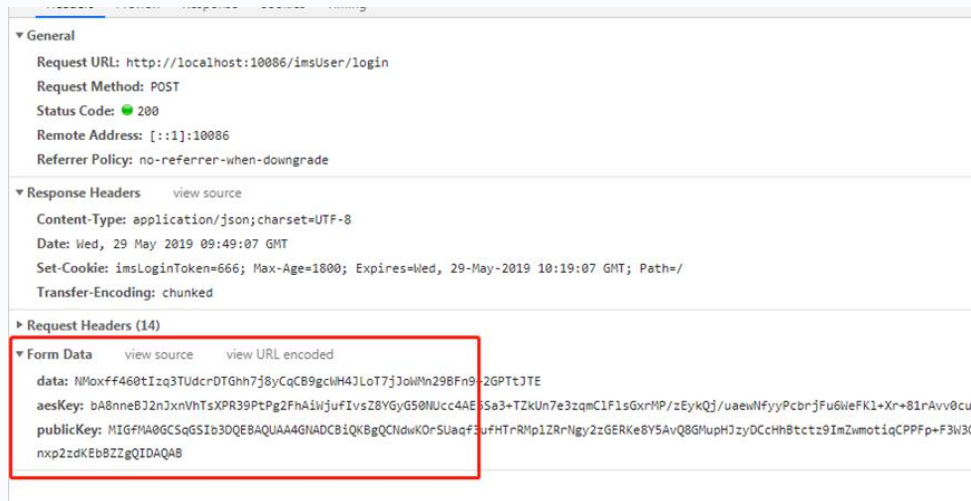
1) 数据加密技术：将一个明文信息经过加密密钥及加密函数转换，变成无意义的、无法直接识别的密文，而接收方则将此密文经过解密函数和解密密钥还原成明文。常见的加密算法有 DES、AES、RSA等。以下以RSA加解密算法作为实例进行演示：

①未加密之前的效果，数据直接暴露在http数据包中：



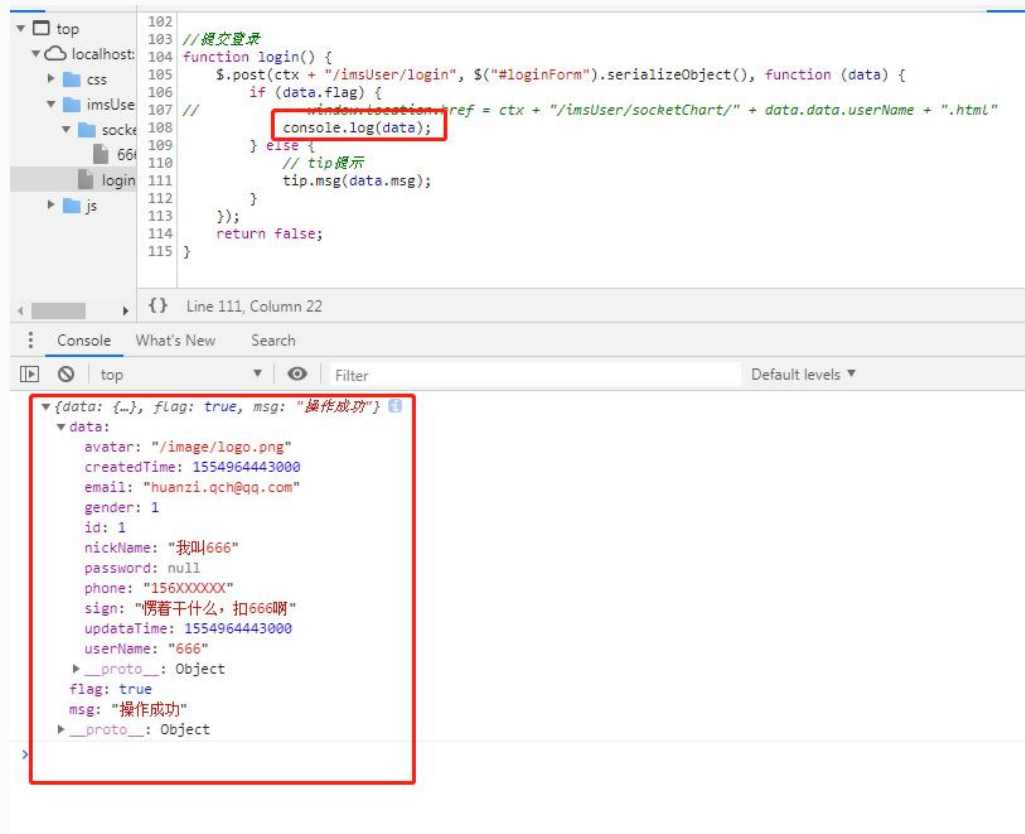
1.3.1 传输信源加密

②加密之后的效果，http中传输的是密文：



1.3.1 传输信源加密

③接收方解密后得到明文：



1.3.1 传输信源加密

2) 哈希算法：其过程是将输入的任意长度的明文信息通过哈希算法转换成固定长度的散列值。通过在数据传输前对数据进行哈希操作，接收方可以根据接收到的数据和相同的哈希函数，来验证数据的完整性和可靠性。一旦数据在传输过程中发生了变化，接收方产生的哈希值就会与发送方计算得到的哈希值不一致，从而激发安全警示。以下以MD5算法作为实例简述哈希算法校验下载文件：

哈希算法中相同的输入永远会得到相同的输出,那么如果输入被修改,输出结果也会大不相同。我们在网站上下载软件的时候，经常看到下载页显示的MD5哈希值：

Windows (x86, 32 & 64-bit), MySQL Installer MSI

Go to Download Page >

Other Downloads:

Windows (x86, 64-bit), ZIP Archive	8.0.29	214.0M	Download
(mysql-8.0.29-winx64.zip)	MD5: 1d0a93f5e6cd89f252693963ef1ab5da		Signature
Windows (x86, 64-bit), ZIP Archive	8.0.29	538.6M	Download
Debug Binaries & Test Suite	MD5: cdbdb54f9332255ca10e659d694a6c25		Signature
(mysql-8.0.29-winx64-debug-test.zip)			

我们只需计算自己本地文件的哈希值,与官网公开的对应文件的哈希值作对比: 如果相同,那么文件下载正确;否则文件有被篡改过。

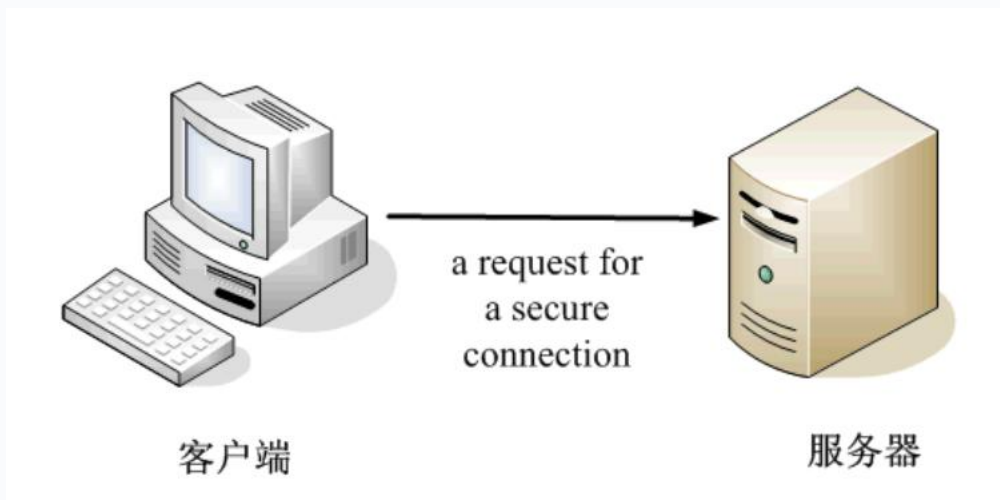
1.3.1 传输信源加密

3) 数字证书技术：数字证书的出现是为了解决如何在计算机网络中识别对方身份的问题的。数字证书由权威机构CA证书授权中心发行，确保了数据在传输过程中的不可抵赖性和不可篡改性。

密钥管理技术：密钥管理是数据加密的核心部分，负责加密密钥的生成、分发和销毁等工作。密钥一般可以分为三级：三级密钥是用来对数据进行加密的密钥，也叫数据加密密钥；二级密钥是用来加密三级密钥的密钥，也叫密钥加密密钥；一级密钥是用来加密二级密钥的密钥，也称为主密钥。密钥管理首先需要生成和存储密钥，然后是分发和管理密钥的服务部分，用于获取生成和储存中的密钥并进行分发管理，最后是提供一些编程接口等。

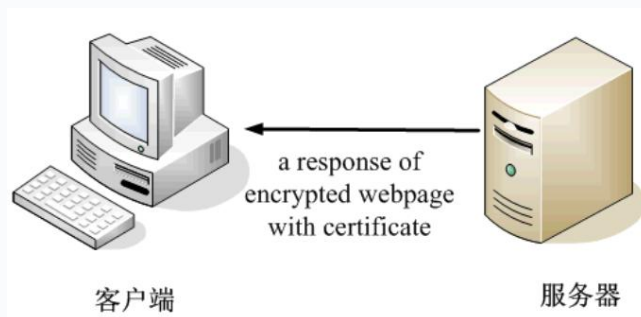
以下以https协议作为实例。这个协议主要用于网页加密。

①首先，客户端向服务器端发送加密请求：

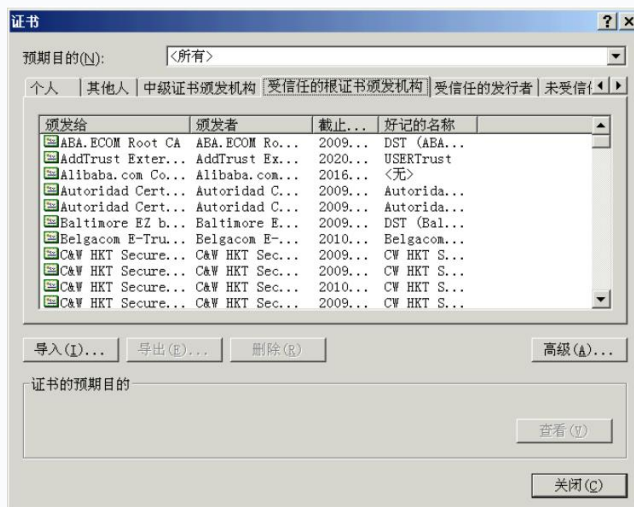


1.3.1 传输信源加密

②服务器用自己的私钥加密网页以后，连同本身的数字证书，一起发送给客户端。



③客户端（浏览器）的"证书管理器"，有"受信任的根证书颁发机构"列表。客户端会根据这张列表，查看解开数字证书的公钥是否在列表之内。

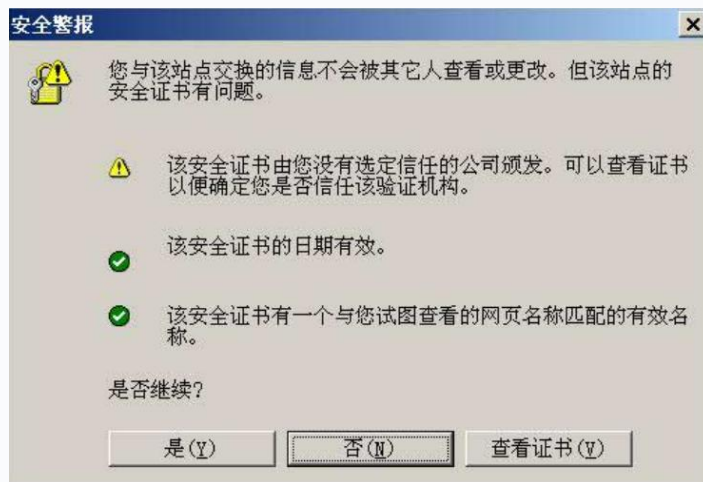


1.3.1 传输信源加密

④如果数字证书记载的网址，与你正在浏览的网址不一致，就说明这张证书可能被冒用，浏览器会发出警告。

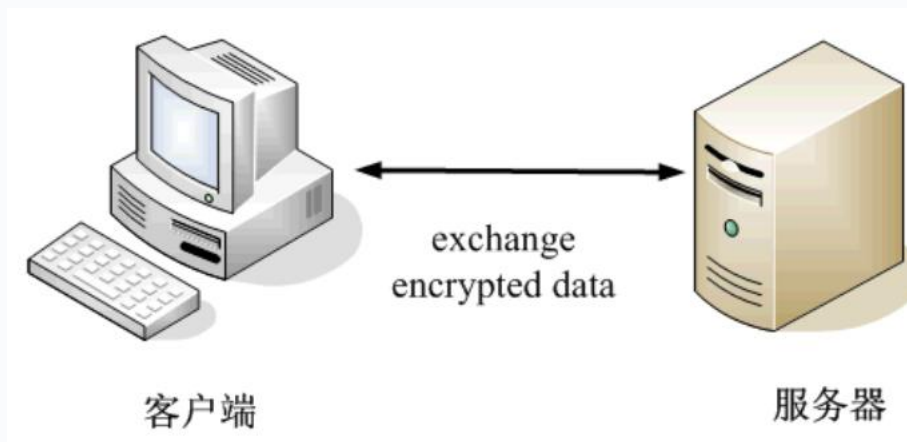


⑤如果这张数字证书不是由受信任的机构颁发的，浏览器会发出另一种警告。



1.3.1 传输信源加密

⑥数字证书如果是可靠的，客户端就可以使用证书中的服务器公钥，对信息进行加密，然后与服务器交换加密信息。



1.3.1 传输信源加密

2) Telnet与SSH

Telnet协议是一种通过TCP/IP建立在可靠传输协议TCP之上的远程登录协议，位于OSI模型的第7层，即应用层。Telnet通过创建虚拟终端来提供与其他服务器的终端仿真连接。该协议要求用户进行认证，通常通过用户名和口令的方式进行，是Internet远程登陆服务的标准协议。使用Telnet协议，本地用户可以将其计算机变成远程主机系统的一个终端。

①对称处理连接的两端，即Telnet协议不强迫客户端从键盘输入，也不强迫客户端在屏幕上显示输出。

②包括一个允许客户端和服务端协商选项的机制，并提供了一组标准选项；

③定义了一个网络虚拟终端，为远程系统提供了一个标准接口。客户端程序无需深入了解远程系统，只需构建使用标准接口的程序；

Telnet远程登陆分4个过程：

①建立连接：本地主机与远程主机建立连接，实际上是创建一个TCP连接。用户需要知道远程主机的IP地址或域名。

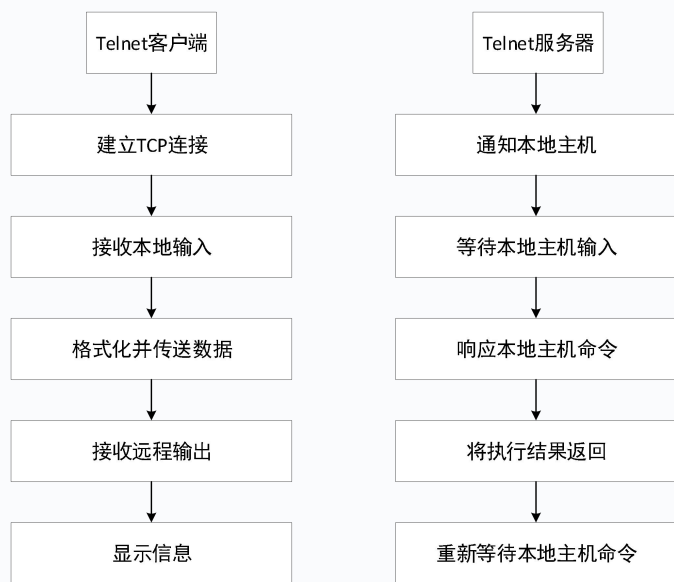
②传输数据：本地终端上输入的用户名、口令以及后续输入的所有命令或字符以NVT（Net Virtual Terminal）格式传送到远程主机。这一过程实际上是从本地主机向远程主机发送一个IP数据包。

③数据转换：远程主机输出的NVT格式的数据被转换为本地终端所接受的格式，并送回本地终端。这包括输入命令的回显和命令执行结果。

④断开连接：最后，本地终端发起断开连接的操作，实质上是撤销TCP连接。

1.3.1 传输信源加密

当我们使用Telnet登陆进入远程计算机系统时，在客户端与服务器上各运行了一个Telnet程序，分别为Telnet客户程序以及Telnet服务器程序，Telnet的C/S模型交互过程如下图所示：



虽然Telnet相对简单、实用且方便，但在极为注重网络安全的现代技术环境中，Telnet并未得到广泛使用。其主要原因在于Telnet作为一种明文传送协议，将用户的所有内容，包括用户名和密码，以明文形式传输于互联网之中，由此带来一定的安全风险。因此，很多服务器选择禁用Telnet服务。如果需要使用Telnet进行远程登录，使用前应当在远端服务器上检查并设置允许Telnet服务的功能。大多数情况下，均以SSH协议代替Telnet协议。

1.3.1 传输信源加密

SSH协议是应用层协议并且是目前较可靠，专为远程登录会话和其他网络服务提供安全性的协议，利用SSH协议可以有效防止远程管理过程中的信息泄露问题，同样的，SSH协议也是建立在可靠的传输协议TCP之上，如下图为SSH的协议框架。



具体流程如下：

- 1) 服务端和客户端各自发送算法协商报文给对方，报文中包含支持的公钥算法列表、加密算法列表、MAC算法列表、压缩算法列表等协商信息。
- 2) 服务端和客户端根据对方和自身支持的算法列表，确定最终使用的算法。双方利用Diffie-Hellman Exchange算法，通过主机密钥对等参数生成会话密钥和会话ID，确保双方拥有相同的会话ID和会话密钥。
- 3) 在后续的数据通信中，服务端和客户端都使用会话密钥进行加密和解密，以确保数据传输的安全性。这样的机制保障了双方在通信过程中的隐私和数据完整性。

因此与Telnet协议相比，在SSH协议中添加了密钥协商阶段，保障了服务器与客户端之间交互的安全性。

1.3.2 通道加密

安全通道是指用于保护数据在传输过程中的一个安全的隧道。它是一种通过在网络上传输数据时确保数据安全性的通道，安全通道的建立保证了不会有第三方干扰或窃取数据。一些常见的安全通道包括VPN、SSH协议、IPsec协议等。

VPN的隧道协议主要分为三种：PPTP、L2TP和IPSec。其中，PPTP和L2TP协议工作在OSI模型的第二层，因此被称为二层隧道协议；而IPSec则属于第三层隧道协议，是最常见的一种。

IPSec并非单一协议，而是一个框架，它由多种协议和功能组合而成，构成了一个完整的VPN方案。这个框架涉及到的内容包括加密算法、验证算法、封装协议、封装模式、密钥有效期等等。

建立IPSec VPN的前提是，在两个站点之间进行IP数据流的安全传输时，需要首先进行协商。这个协商过程确定了加密算法、封装技术以及密钥等参数。协商过程分为两个阶段：第一个阶段用于建立管理连接，而第二个阶段则用于建立数据连接。

1.3.2 通道加密

阶段一旨在在两个对等设备之间建立一个安全的管理连接，该连接并不传输实际的数据，而是用于保护第二阶段的协商过程。在阶段一中，需要进行以下协商：双方需要使用什么加密算法进行加密（DES、AES...）

①加密算法选择：双方需要确定使用哪种加密算法进行数据加密，例如DES、AES等。

②摘要认证方式：确定采用哪种摘要认证方式来保证数据的完整性，常见的有MD5、SHA等。

③密钥共享方式：确定双方采用的密钥共享方式，可以是预共享密钥、CA数字签名、或者公钥认证等。

④密钥强度选择：确定使用的密钥强度，通常密钥越长，加密强度越高。

⑤管理链接生存时间：设置管理链接的生存时间，通常以秒为单位，默认为一天

⑥协商模式选择：确定协商模式，可以是主模式或者积极模式。

这些协商内容将确保在第二阶段的数据连接建立过程中，双方能够安全地传输数据，并保证数据的机密性、完整性和可用性。

1.3.2 通道加密

阶段二是在安全的管理连接建立后，对等体之间协商用于构建安全数据连接的安全参数。这个协商过程是安全的且加密的，协商完成后，将在两个站点之间建立安全的数据连接。阶段二中需要协商的内容：

①传输模式：确定使用的传输模式，可以是隧道模式（Tunnel Mode）或传输模式（Transport Mode）。**隧道模式将整个IP数据报都加密并封装在新的IP数据报中，适用于整个IP数据报需要保护的情况；传输模式只对IP数据报的有效载荷进行加密，适用于端到端的通信。**

②封装技术：**确定用于封装和加密数据的技术，通常有ESP（Encapsulating Security Payload）和AH（Authentication Header）两种。ESP提供加密和认证功能，适用于保护数据的完整性和保密性；AH提供数据的完整性认证，但不提供加密功能。**

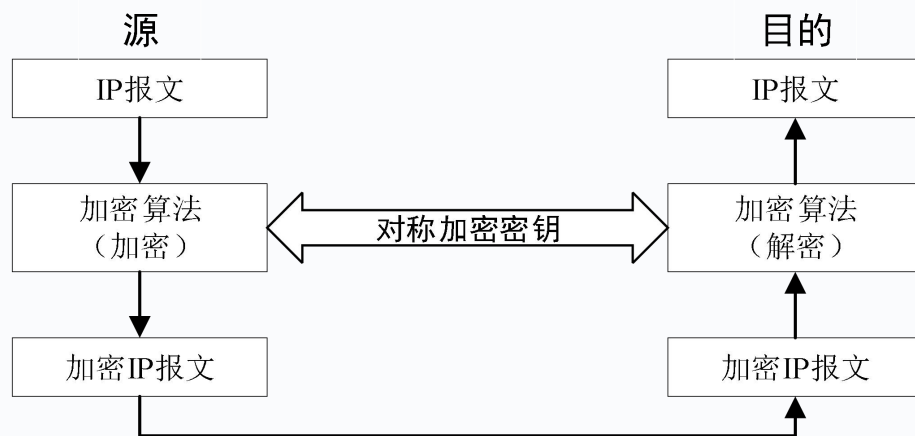
③数据加密方式：确定在传输过程中所采用的数据加密算法，常见的有DES、3DES、AES等。加密算法的选择应考虑安全性和性能要求

④数据认证方式：确定传输过程中所采用的数据认证算法，用于验证数据的完整性，通常使用MD5或SHA等哈希算法。

⑤可以定义需要使用IPSec保护的流量，包括源IP地址、目标IP地址、端口号等信息，以确定哪些数据需要受到IPSec保护。

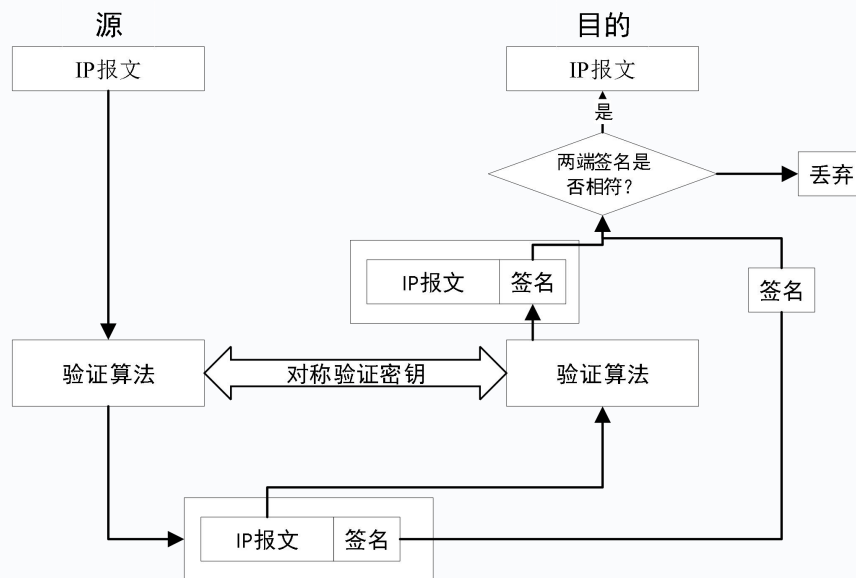
1.3.2 通道加密

一方传递消息之前，先使用加密算法和加密密钥；另一方收到消息后，使用相同的加密算法和加密密钥。对称加密算法，主要包括DES、3DES和AES。



1.3.2 通道加密

验证算法在信息传递中起到了重要作用。一方在传递消息之前，会使用验证算法和验证密钥对消息进行处理，生成签名以确保消息的完整性和真实性。这个签名相当于文书上的签字画押，随消息一同发送出去。另一方收到消息后，同样使用相同的验证算法和验证密钥对消息进行处理，得到签名，并将其与发送方提供的签名进行比对。如果两端的签名一致，就证明该消息没有被篡改。常用的验证算法包括MD5和SHA系列，它们能够提供有效的数据完整性保护。如下图为验证流程：



1.3.2 通道加密

封装协议AH (Authentication Header) 是一种基于IP的传输层协议，其协议号为51。AH协议主要提供数据的认证功能，但不支持数据的加密。它对整个IP数据报的头部进行认证，以确保数据在传输过程中的完整性和真实性。

ESP协议 (Encapsulating Security Payload) 也是基于IP的传输层协议，其协议号为50。ESP协议支持数据的加密和认证功能。

在前面部分介绍了AH和ESP两种IPSec协议的工作原理和功能。现在让我们来探讨IPSec的封装：

封装模式：封装模式是指将AH或ESP相关的字段插入到原始IP报文中，以实现报文的认证和加密，封装模式有传输模式和隧道模式两种。当安全协议同时采用AH和ESP时，AH和ESP协议必须采用相同的封装模式。

①**传输模式：**在传输模式中，AH头或ESP头被插入到IP头与传输层协议头之间，用于保护TCP/UDP/ICMP等传输层协议的负载数据。传输模式下，报文头不会被改变，而是对数据负载进行加密和认证。AH协议的完整性验证范围为整个IP报文，这意味着整个IP数据包（包括IP头部和数据部分）都会被验证。而ESP协议在传输模式下无法保证IP头的安全，即IP头部信息不受加密或认证的保护。

1.3.2 通道加密

原始报文

IP Header	TCP Header	data
-----------	------------	------

AH封装报文

IP Header	AH	TCP Header	data
-----------	----	------------	------

ESP封装报文

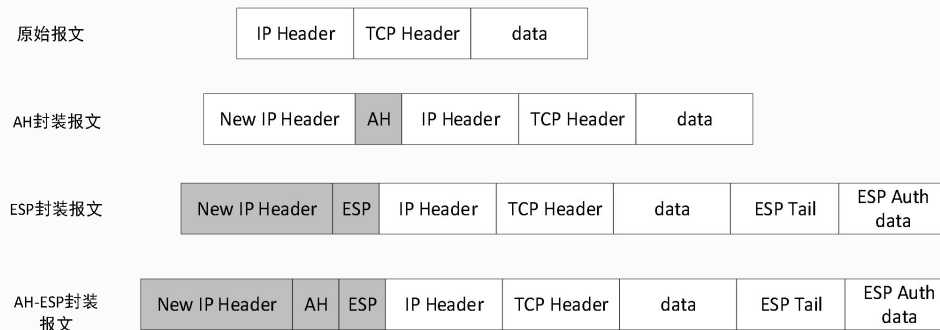
IP Header	ESP	TCP Header	data	ESP Tail	ESP Auth data
-----------	-----	------------	------	----------	---------------

AH-ESP封装
报文

IP Header	AH	ESP	TCP Header	data	ESP Tail	ESP Auth data
-----------	----	-----	------------	------	----------	---------------

②隧道模式：在隧道模式中，会在原始IP头部之前插入ESP或AH头部，并生成新的IP头部。换句话说，隧道模式会对整个原始IP数据包进行封装，包括原始IP头部和数据部分，并在新的IP头部中添加了额外的信息。这种方式使得数据在传输过程中被保护，并且可以用于网络之间的通信，特别适用于两个网关之间的通信。因此，隧道模式是一种常用的封装模式，它能够提供高效的数据传输和安全保护。

1.3.2 通道加密



传输模式和隧道模式在IPSec中有着不同的运用。在传输模式下，AH和ESP处理前后IP头部保持不变，更适用于直接的点对点通信。而隧道模式则会在AH和ESP处理后，添加一个新的IP头部，适用于跨网络传输，比如网关之间的通信。

IPSec检测机制包括Keepalive机制和DPD机制。

Keepalive机制通过IKE对等体定期发送Hello消息来确认其活动状态。如果在超时时间内未收到Hello消息，则将对端视为不可达，并删除相应的IKE SA和IPSec SA。默认情况下，Keepalive机制处于关闭状态，需要根据需求配置相应参数以使其生效。

相比之下，DPD（Dead Peer Detection）机制不会周期性发送Hello消息，而是通过IPSec流量来最小化状态检测所需的消息数量。只有在一定时间内未收到对端的流量时，才会发送DPD报文来检测对端状态。如果连续几次未收到对端的响应，则认为对端不可达，并删除相应的IKE SA和IPSec SA。

1.3.2 通道加密

在实际应用中，Keepalive机制使用较少的原因主要有两点：

- ①周期性发送Hello消息会消耗大量CPU资源，限制了可建立的IPSec会话数量。
- ②Keepalive缺乏统一标准，各厂商设备可能无法兼容。

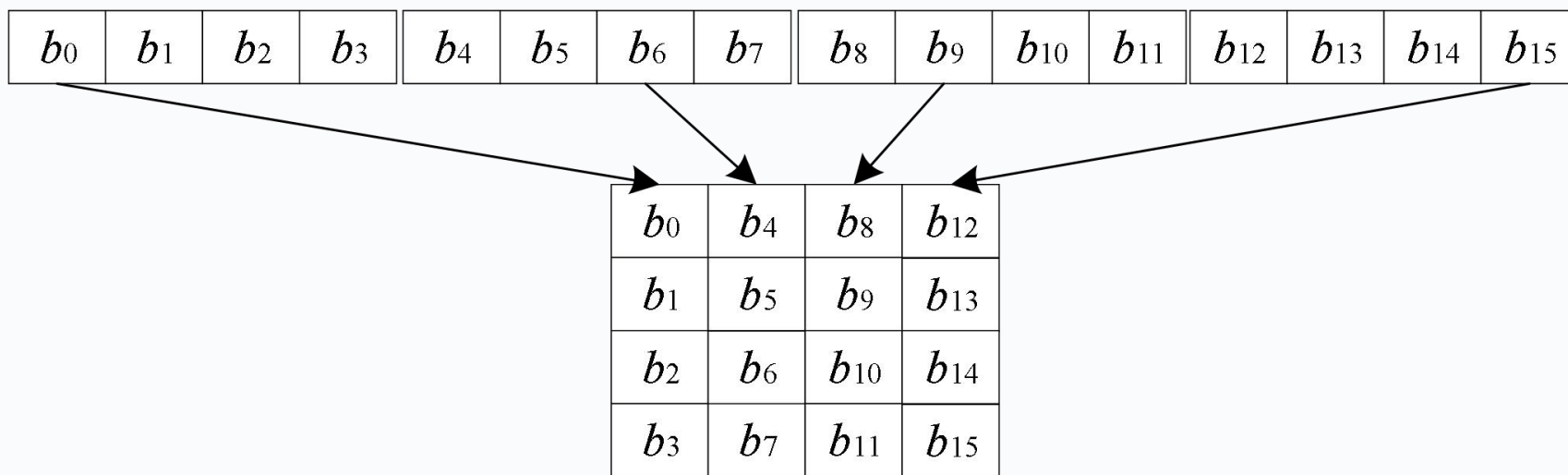
总之，数据传输过程中的机密性保证是为了保证数据从发送方到接收方正确接收信息的安全，其主要目的是实现数据加密、防篡改和高可用。数据在传输过程中的安全性是保护个人隐私和商业机密的重要环节。通过采用加密算法、数字证书以及安全协议等技术手段，确保在数据传输过程中数据的机密性，并减轻安全威胁所带来的风险。



谢谢！

1.1.2 密码算法之AES

AES算法中初始状态矩阵由1组长度为128比特明文分组构成，以字节为单位，则总共有16字节，记为 b_0 、 b_1 、 b_2 b_{15} 。AES的算法明文分组可以构成一个 4×4 的初始矩阵，如下图所示：



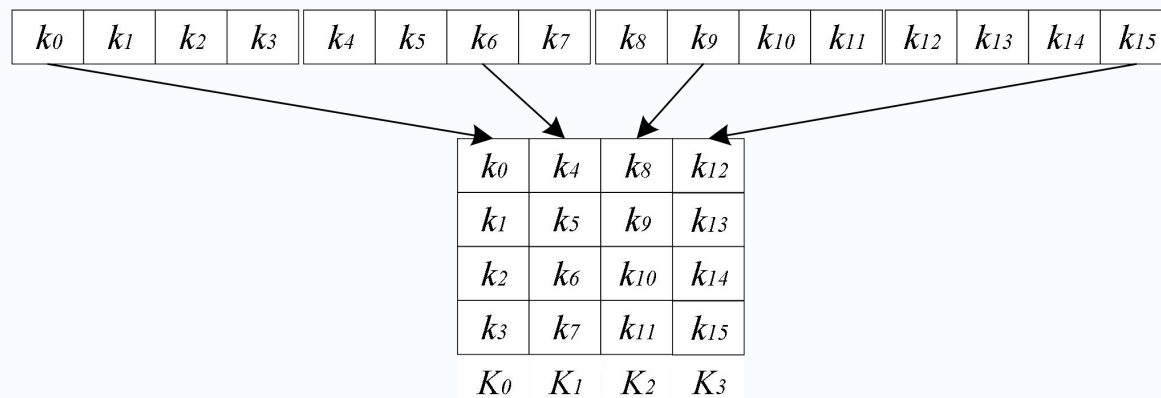
1.1.2 密码算法之AES

AES算法中的密钥以字节为单位进行计算，密钥列数记为，密钥状态也是4行，每列4字节数据，即32比特。如下式：

$$N_k = \frac{\text{密钥长度(bit)}}{32}$$

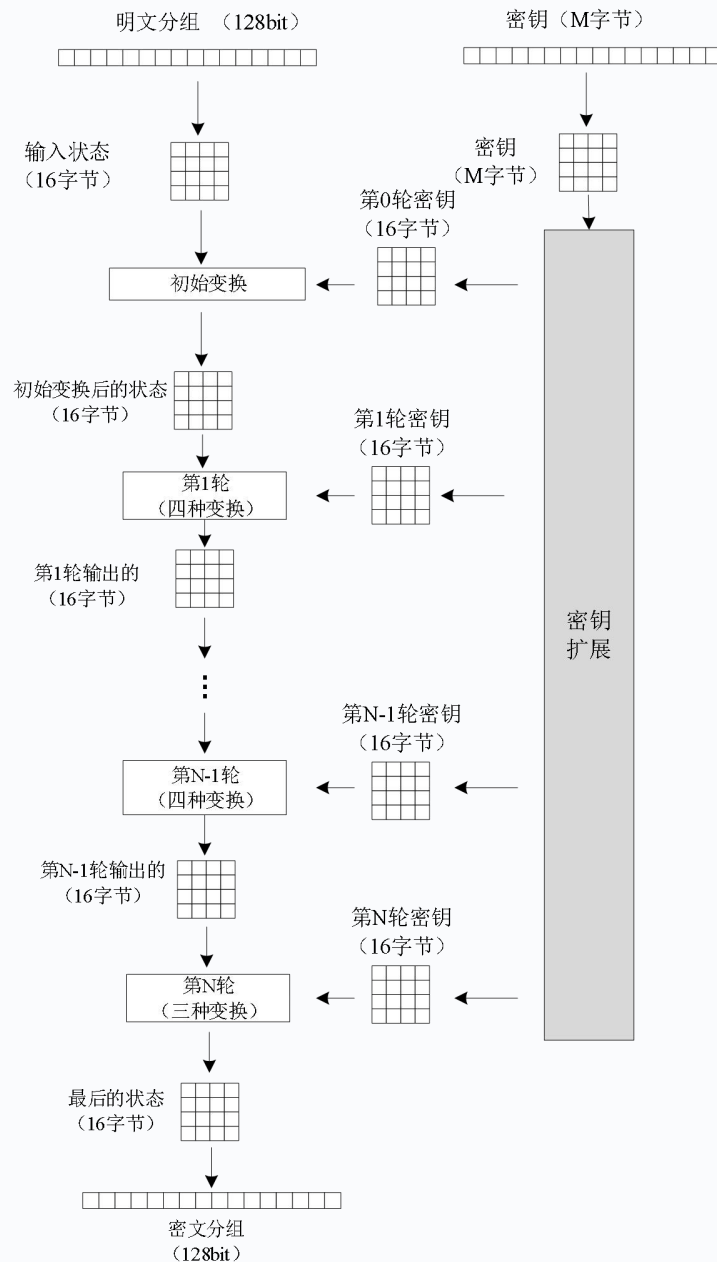
1.1.2 密码算法之AES

AES算法的密钥长度的列数为128比特，192比特，256比特，因此密钥列数 也可分为4，6，8。假设我们的密钥长度为128比特，以字节为单位，则总共有16字节，记为 k_0 、 k_1 、 k_2 k_{15} 。类似的每4字节为1列依次排序，总共4列，每列子密钥记为 K_0 、 K_1 、 K_2 、 K_3 。如下图：



1.1.2 密码算法之AES

AES加密算法的加密流程由多轮加密构成，除了结尾的一轮，其他轮都是由四个步骤组成：字节代替、行移位、列混合、轮密钥加。而最后一轮仅包括字节代替、行移位、轮密钥加这三步。轮数与密钥的长度相关，16字节的密钥对应着迭代10轮，24字节的密钥对应着迭代12轮，32字节的密钥对应着迭代14轮。



1.1.2 密码算法之AES

AES算法每一个步骤都是可逆的，因此只要把AES加密中的每一步换成其逆变换，即可得到AES解密算法。值得注意的是，解密算法也是先进行第0轮，而且最后一轮也是只有三步。

接下来逐步讲解字节替代、行移位、列混合、轮密钥加、密钥扩展以及解密过程。

1.1.2 密码算法之AES

AES算法每一个步骤都是可逆的，因此只要把AES加密中的每一步换成其逆变换，即可得到AES解密算法。值得注意的是，解密算法也是先进行第0轮，而且最后一轮也是只有三步。

接下来逐步讲解字节替代、行移位、列混合、轮密钥加、密钥扩展以及解密过程。

1.1.2 密码算法之AES

AES算法每一个步骤都是可逆的，因此只要把AES加密中的每一步换成其逆变换，即可得到AES解密算法。值得注意的是，解密算法也是先进行第0轮，而且最后一轮也是只有三步。

接下来逐步讲解字节替代、行移位、列混合、轮密钥加、密钥扩展以及解密过程。

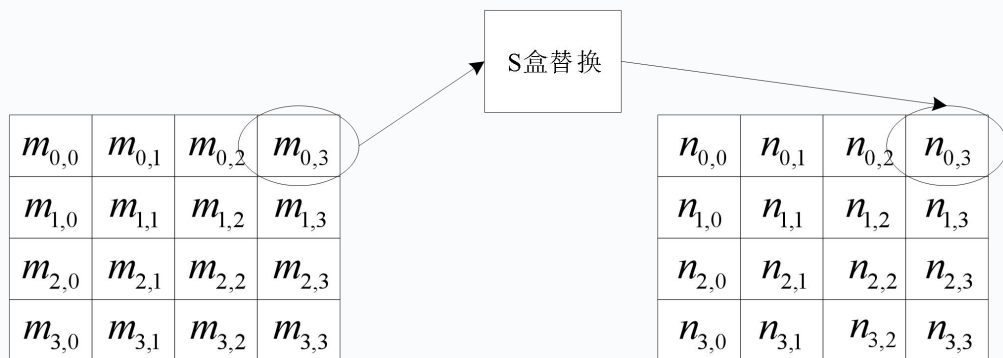
1.1.2 密码算法之AES

1) 字节替代

字节替代将输入的状态矩阵的每一个字节通过查表将其映射为另外一个字节，该表大小为 16×16

查表方法：输入字节的前4比特指定S盒的行值，后4比特指定S盒的列值，比如输入明文为（十六进制）：“85”，那么根据上述方法第8行第5列，对应的值为“97”。

X \ Y	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	6A	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16



1.1.2 密码算法之AES

2) 行移位

每一行循环左移的偏移量由查表可得：由表可知，每1行循环左移的偏移量由明文分组的大小和所在行数共同决定的，AES算法中列数固定为4，即第一行循环左移0字节，第二行循环左移1字节，第三行循环左移2字节，第四行循环左移3字节。

列数	第一行	第二行	第三行	第四行
4	0	1	2	3
6	0	1	2	3
8	0	1	3	4

我们假设有一组 4×4 的状态矩阵作为行移位的输入，并且以第一列的元素作为基准，进行行移位的结果为：

$a_{0,0}$	$a_{0,1}$	$a_{0,2}$	$a_{0,3}$
$a_{1,0}$	$a_{1,1}$	$a_{1,2}$	$a_{1,3}$
$a_{2,0}$	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$
$a_{3,0}$	$a_{3,1}$	$a_{3,2}$	$a_{3,3}$

行移位 $\longrightarrow$

$a_{0,0}$	$a_{0,1}$	$a_{0,2}$	$a_{0,3}$
$a_{1,1}$	$a_{1,2}$	$a_{1,3}$	$a_{1,0}$
$a_{2,2}$	$a_{2,3}$	$a_{2,0}$	$a_{2,1}$
$a_{3,3}$	$a_{3,0}$	$a_{3,1}$	$a_{3,2}$

1.1.2 密码算法之AES

3) 列混合

列混合是将输入的状态矩阵的每列与固定的多项式 $a(x)$ 在有限域 $GF(2^8)$ 上相乘, 然后模 $x^4 + 1$, 其中多项式 $a(x) = \{03\}x^3 + \{01\}x^2 + \{01\}x + \{02\}$ 。混合操作方式如下式所示:

$$c(x) = a(x) \otimes b(x) = a(x) \times b(x) \bmod (x^4 + 1)$$

列混合可用矩阵相乘来实现:

$$\begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \cdot \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

1.1.2 密码算法之AES

4) 轮密钥加

轮密钥加是将列混合的输出矩阵与子密钥进行异或运算，如下图所示：

$$\begin{array}{|c|c|c|c|} \hline c_{0,0} & c_{0,1} & c_{0,2} & c_{0,3} \\ \hline c_{1,0} & c_{1,1} & c_{1,2} & c_{1,3} \\ \hline c_{2,0} & c_{2,1} & c_{2,2} & c_{2,3} \\ \hline c_{3,0} & c_{3,1} & c_{3,2} & c_{3,3} \\ \hline \end{array} \oplus \begin{array}{|c|c|c|c|} \hline k_{0,0} & k_{0,1} & k_{0,2} & k_{0,3} \\ \hline k_{1,0} & k_{1,1} & k_{1,2} & k_{1,3} \\ \hline k_{2,0} & k_{2,1} & k_{2,2} & k_{2,3} \\ \hline k_{3,0} & k_{3,1} & k_{3,2} & k_{3,3} \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline d_{0,0} & d_{0,1} & d_{0,2} & d_{0,3} \\ \hline d_{1,0} & d_{1,1} & d_{1,2} & d_{1,3} \\ \hline d_{2,0} & d_{2,1} & d_{2,2} & d_{2,3} \\ \hline d_{3,0} & d_{3,1} & d_{3,2} & d_{3,3} \\ \hline \end{array}$$

在轮密钥加的过程中，子密钥状态矩阵的长度应等于明文状态矩阵长度。而具体轮密钥加的过程则见下式：

$$c_{i,j} \oplus k_{i,j} = d_{i,j}$$

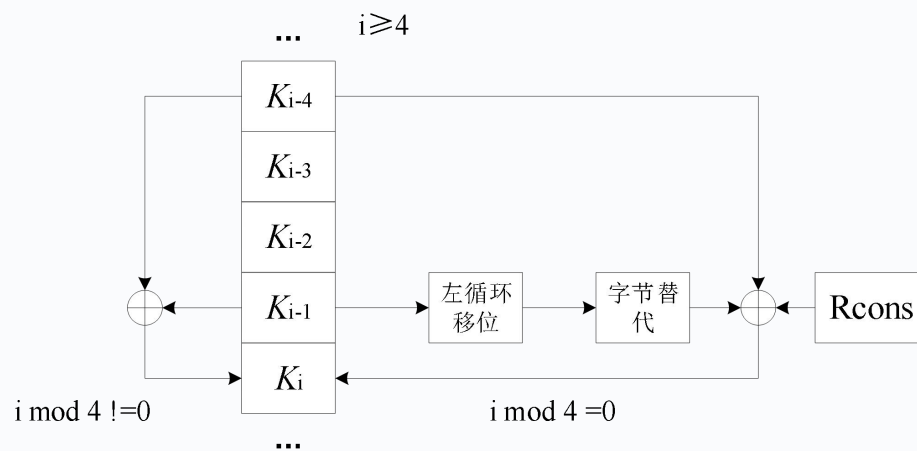
1.1.2 密码算法之AES

5) 密钥扩展

在AES算法中，初始密钥会被扩展成多个轮密钥，这些轮密钥在每一轮与明文进行加密。密钥比特=明文分组长度 $\times$ (轮数+1),密钥长度，轮数、以及轮密钥长度关系如下表：

密钥长度 (比特)	轮数	轮密钥长度 (比特)
128	10	1408
192	12	2496
256	14	3840

轮密钥生成过程主要步骤：将最后一个字作为临时字并将其左移一个字节，对临时字的每个字节进行字节替代变换，将临时字与上一个轮密钥进行异或操作，并将结果当作当前轮密钥，对当前轮密钥进行列混合变换，但不包括最后一个轮密钥。128比特密钥扩展如右图所示：



1.1.2 密码算法之AES

Rcons是常量

当 i 是4的倍数时: $K_i = K_{i-4} \oplus K_{i-1}$,

当 i 不是4的倍数时: $K_i = K_{i-4} \oplus T(K_{i-1})$,

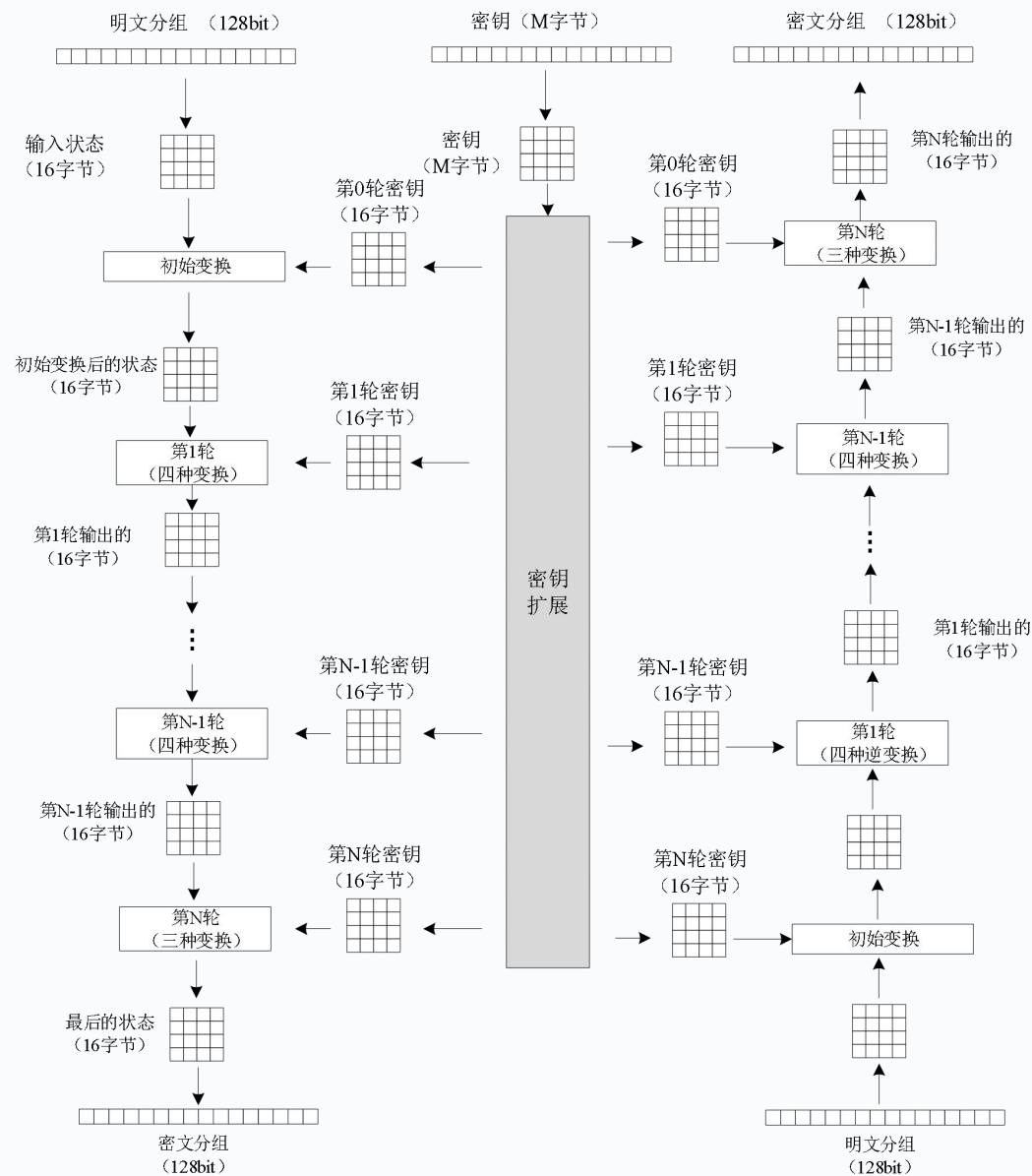
将列以字节为单位循环左移1字节, 然后将循环左移后的输出利用字节替代, 根据下列Rcon常量表, 得到 $Rcon[i/(密钥比特数/32)]$ 的值, 最后将字节替代的值与 $Rcon[i/(密钥比特数/32)]$ 进行异或。

i	1	2	3	4	5
Rcon[i]	01000000	02000000	04000000	08000000	10000000
i	6	7	8	9	10
Rcon[i]	20000000	40000000	80000000	1b000000	36000000

1.1.2 密码算法之AES

6) 解密过程

AES的解密过程就是逆字节替代，逆行移位以及逆列混合。如由图所示：



1.1.2 密码算法之AES

6) 解密过程

①逆字节替代

逆S盒表与S盒表的运用一致，输入字节的前4比特为行值，后4比特为列值。

Y X	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
5	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D

1.1.2 密码算法之AES

6) 解密过程

② 逆行移位

与行移位相反，逆行移位就是第一行不移位，第二行循环右移1位，第二行循环右移2位，第三行循环右移3位。



③ 逆列混合

b_0
b_1
b_2
b_3

=

0E	0B	0D	09
09	0E	0B	0D
0D	09	0E	0B
0B	0D	09	0E

·

a_0
a_1
a_2
a_3

1.1.3 密码算法之RSA

1) 算法概述

①密钥产生

选择两个满足需要的大素数 p 和 q , 计算 $n = p \times q$, $\varphi(n) = (p-1) \times (q-1)$, 其中 $\varphi(n)$ 是 n 的欧拉函数。

选择一个整数 e , 满足 $1 < e < \varphi(n)$, 通过 $d \times e \equiv 1 \pmod{\varphi(n)}$, 计算出 d 。

以 $\{e, n\}$ 为公开密钥, $\{d, n\}$ 为私密密钥。

现有A方接收外界的消息, 则外界知道A的公开密钥 $\{e, n\}$, 只有A知道自己的私密密钥 $\{d, n\}$ 。

②加密

若发送方想要利用RSA算法保护其欲发送给A方的消息 m , 利用A公布给外界的公开密钥 $\{e, n\}$, 加密, 计算 $c \equiv m^e \pmod{n}$, 然后将密文 c 发送给A

③解密

A方接收到密文 c 之后, 根据自己的私钥对 c 进行解密: $m = c^d \pmod{n}$ 。所得到的结果 m 即为发送方想要发送的消息 m 。

1.1.3 密码算法之RSA

2) 算法正确性证明

我们已知: $c^d \equiv m^e \pmod{n} \equiv m^{1 \bmod \varphi(n)} \pmod{n} \equiv m^{k\varphi(n)+1} \pmod{n}$

① m 与 n 互素, 因此由Euler定理得: $m^{\varphi(n)} \equiv 1 \pmod{n}$, 则 $m^{k\varphi(n)} \equiv 1 \pmod{n}$, 故 $m^{k\varphi(n)+1} \equiv m \pmod{n}$, 则 $c^d \pmod{n} \equiv m$ 。

② m 与 n 不互素且 $m < n$, $\gcd(m, n) \neq 1$, m 不可能是 q 和 p 的倍数, 设 $m = cp$, 其中 c 为正整数。此时 $\gcd(m, q) = 1$, 由Euler定理得: $m^{\varphi(q)} \equiv 1 \pmod{q}$ 。所以:

$$m^{k\varphi(q)} \equiv 1 \pmod{q}$$

$$[m^{k\varphi(q)}]^{\varphi(p)} \equiv 1 \pmod{q}$$

$$m^{k\varphi(n)} \equiv 1 \pmod{q}$$

因此存在一整数 r , 使得 $m^{k\varphi(n)} = 1 + rq$, 两边乘 $m = cp$ 得:

$$m^{k\varphi(n)+1} = m + rcpq = m + rcn$$

即 $m^{k\varphi(n)+1} \equiv m \pmod{n}$, 所以可得 $c^d \pmod{n} \equiv m$

1.1.4 密码算法之MD5

1) 算法概述

①填充消息

初始为输入消息为 M ，我们需要保证MD5算法处理消息的总长度为512比特的倍数。那么现在假设消息 M 得长度为 i 比特，那么填充规则是在原始消息 M 的尾部添加1比特的1和 j 比特(为非负整数)的0,满足 $i + 1 + j = 448(\text{mod } 512)$ ，填充完的消息会是 $n-1$ 个512比特的块和一个448比特的块，最后需要在末尾添加代表原始消息变换为二进制长度的64比特块，若长度小于64比特，在左侧填充0式其长度达到64比特。这样整体消息便由 n 个512比特的块组成。

②被填充消息分组

在对原始输入消息 M 进行填充后得到 M' ，将 M 进行512比特的分组，在上一步的填充消息步骤中，我们能保证 M' 由 n 个512比特的块组成。因此将划分的块依次表示为：

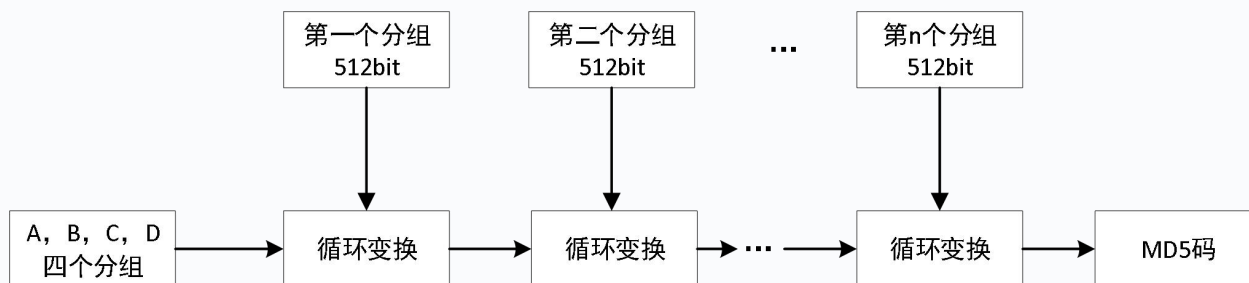
$M^{(1)}, M^{(2)}, M^{(3)}, \dots, M^{(n)}$ 。

每一个长度为512比特的块，我们由按照32比特划分，每个可以划分出16个块，假设将第一个块 $M^{(1)}$ 以32比特划分，我们以 $M_1^{(1)}, M_2^{(2)}, M_3^{(3)}, \dots, M_{16}^{(1)}$ 分别进行标识。

1.1.4 密码算法之MD5

1) 算法概述

③MD5算法处理



1.1.4 密码算法之MD5

1) 算法概述

假设MD5算法的初始变量为：

$$E_0^{(0)} = 67452301$$

$$E_1^{(0)} = EFCDAB89$$

$$E_2^{(0)} = 98BADCFE$$

$$E_3^{(0)} = 10325476$$

总计128比特，将四个块分别存储在A, B, C, D四个寄存器，每个分块一共四个回合，每回合计算16步，合计64轮次。随后将用于保存Hash函数的中间结果。在每个16步之后，得到128比特输出再与输入 $E^{(i-1)}$ 进行进行模 2^{32} 加操作，得到的分块处理最终结果即为128比特的数据块 $E^{(i)}$ ，每一个32比特子分组的变换如下图所示：

1.1.4 密码算法之MD5

1) 算法概述

图中F函数表示基本逻辑函数，但是F函数是包含了四个结构相似的四个基本逻辑函数并且每个回合使用不同的逻辑函数（ $\wedge$ 表示按位与， $\vee$ 表示按位或， $\oplus$ 表示异或， $\neg$ 表示非运算）：

$$f_1 = (t, B, C, D) = (B \wedge C) \vee (\neg B \wedge D)$$

$$0 \leq t \leq 15$$

$$f_2 = (t, B, C, D) = (B \wedge D) \vee (C \wedge \neg D)$$

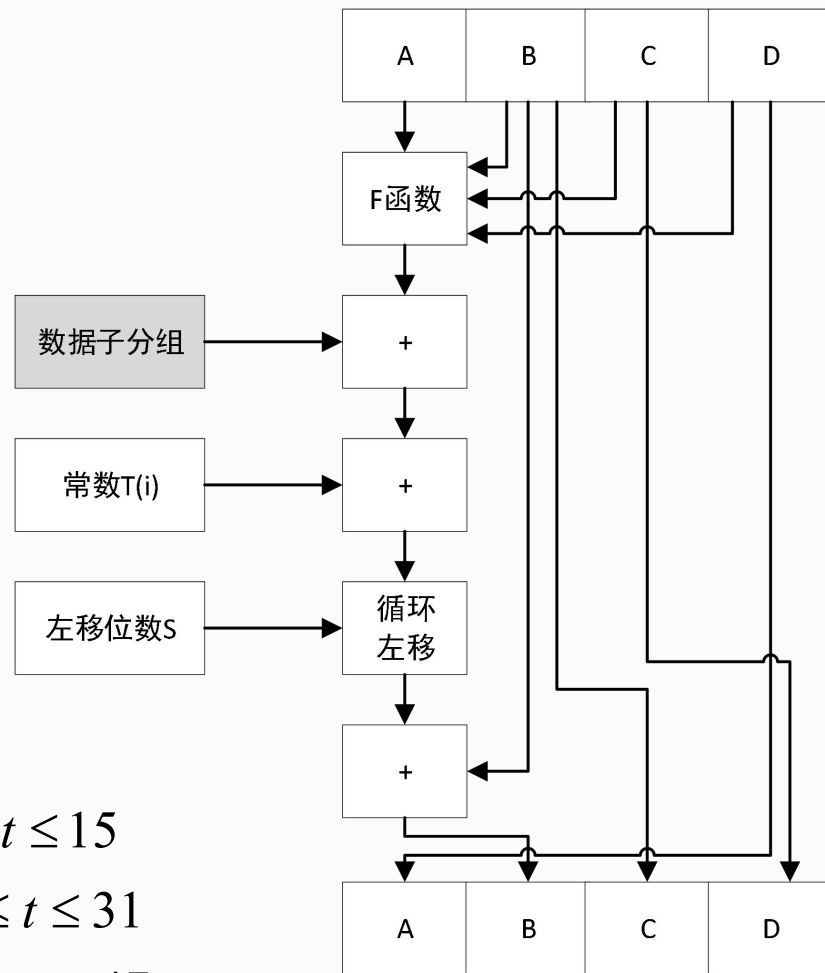
$$16 \leq t \leq 31$$

$$f_3 = (t, B, C, D) = B \oplus C \oplus D$$

$$32 \leq t \leq 47$$

$$f_4 = (t, B, C, D) = C \oplus (B \wedge \neg D)$$

$$48 \leq t \leq 63$$



1.1.4 密码算法之MD5

1) 算法概述

数据子分组长为32比特，是由512比特的划分而来，划分方法如下所示（ t 表示轮次， i 表示第 i 个数据块）：

$$\text{数据子分组} = \begin{cases} M_t^{(i)} & 0 \leq t \leq 15 \\ M_{(1+5(t-16)) \bmod 16}^{(i)} & 16 \leq t \leq 31 \\ M_{(5+3(t-32)) \bmod 16}^{(i)} & 32 \leq t \leq 47 \\ M_{(7(t-48)) \bmod 16}^{(i)} & 48 \leq t \leq 63 \end{cases}$$

1.1.4 密码算法之MD5

1) 算法概述

MD5有64个常数，由sin函数构成，常量表达式为： $T(i) = \text{int}(4294967296 | \sin(i) |)$ ，其中i表示第i此变换。

轮次	阶段															
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	7	12	17	22	7	12	17	22	7	12	17	22	7	12	17	22
2	5	9	14	20	5	9	14	20	5	9	14	20	5	9	14	20
3	4	8	16	23	4	8	16	23	4	8	16	23	4	8	16	23
4	6	10	13	21	6	10	15	21	6	10	15	21	6	10	15	21

当16个子分组处理完之后，即完成了一轮循环变换，一共需要四轮，四轮得到的值再分别与原值相加，就可以得到A，B，C，D的结果，最后再将其以十六进制拼接起来，就能得到MD5的值。

1.1.5 密码算法之SHA1

1) 算法概述

SHA1的初始化变量与MD5不同的是，其初始变量为160比特的数据块，即5个32比特的字，依次为 $H_0^{(0)}$, $H_1^{(0)}$, $H_2^{(0)}$, $H_3^{(0)}$, $H_4^{(0)}$ ，设置如下：

$$H_0^{(0)} = 67452301$$

$$H_1^{(0)} = EFCDAB89$$

$$H_2^{(0)} = 98BADCFE$$

$$H_3^{(0)} = 10325476$$

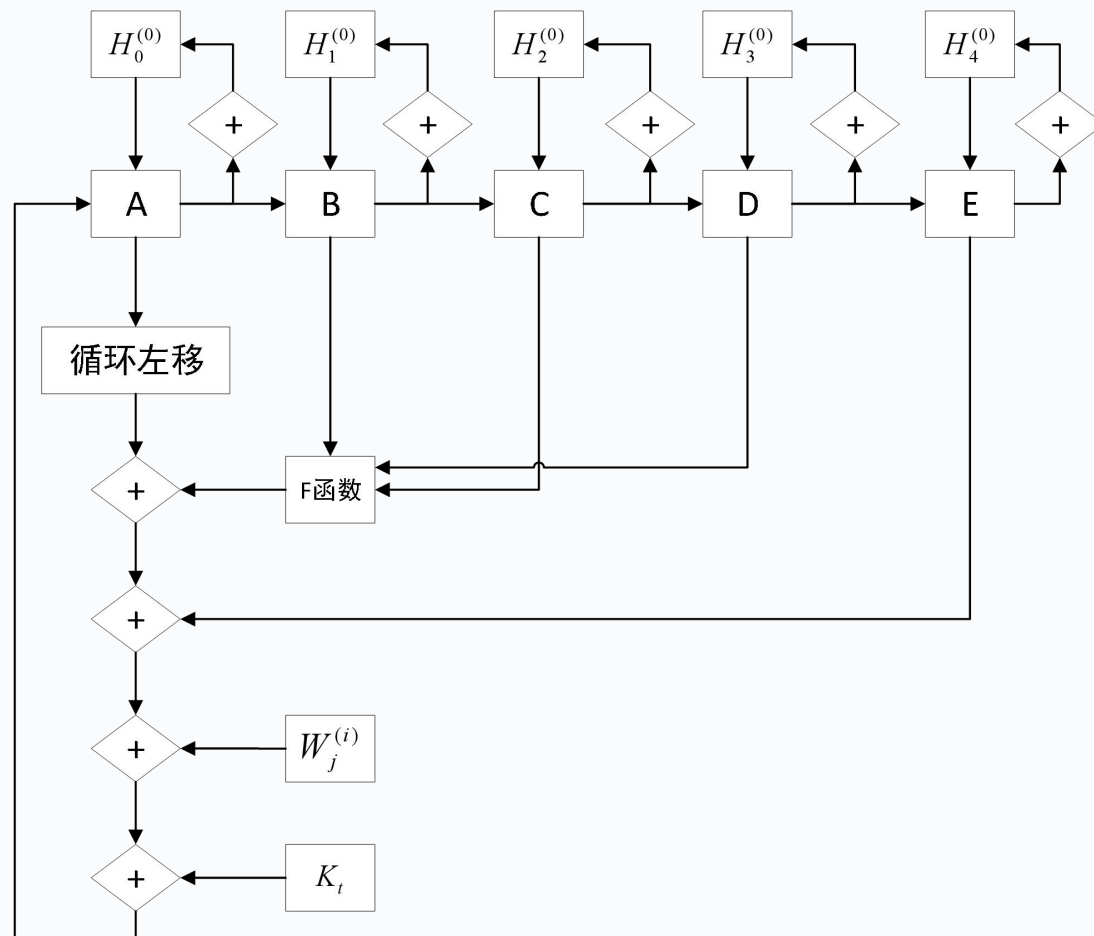
$$H_4^{(0)} = C3D2E1F0$$

1.1.5 密码算法之SHA1

1) 算法概述

SHA1包含4个回合运算，每一回合20步，总共80步，算法流程如右所示：

如上图，SHA1算法将充分块后 i 个512比特的 $M^{(i)}$ 划分16个32比特的块，依次表示为 $W_1^{(i)}, W_2^{(i)}, \dots, W_{16}^{(i)}$ ，先给五个寄存器A, B, C, D, E赋初值，这些寄存器最后都会用于保存Hash函数的中间结果和最终结果。



1.1.5 密码算法之SHA1

1) 算法概述

因为轮次 t 的不同，F函数的定义也与MD5中不同:

$$\begin{aligned} f_1(t, B, C, D) &= (B \wedge C) \vee (B \wedge D) & 0 \leq t \leq 19 \\ f_2(t, B, C, D) &= B \oplus C \oplus D & 20 \leq t \leq 39 \\ f_3(t, B, C, D) &= (B \wedge C) \vee (B \wedge D) \vee (C \wedge D) & 40 \leq t \leq 59 \\ f_4(t, B, C, D) &= B \oplus C \oplus D & 60 \leq t \leq 79 \end{aligned}$$

常量 K_t 定义如下:

回合	步骤	输入常量	取整方式 (整数)
1	$0 \leq t \leq 19$	$K_t = 5A82799$	$2^{30} \times \sqrt{2}$
2	$20 \leq t \leq 39$	$K_t = 6ED9EBA1$	$2^{30} \times \sqrt{3}$
3	$40 \leq t \leq 59$	$K_t = 5F1BBCDC$	$2^{30} \times \sqrt{5}$
4	$60 \leq t \leq 79$	$K_t = CA62C1D6$	$2^{30} \times \sqrt{10}$

1.1.5 密码算法之SHA1

1) 算法概述

数据扩展：分块处理中还需要使用 $W_j^{(i)}$ ，定义如下（ $ROLT^i$ 代表循环左移*i*比特）：

$$W_j^i = \begin{cases} W_j^i & 0 \leq t \leq 15 \\ ROLT^1(W[t-3] \oplus W[t-8] \oplus W[T-14] \oplus W[T-16]) & 16 \leq t \leq 79 \end{cases}$$

对于 $0 \leq t \leq 79$ ，每经过一次压缩函数，执行以下循环：

$$T = ROLT^5(A) + f_t(B, C, D) + E + W[t] + K_t \pmod{2^{32}}$$

$$E = D, D = C, B = A, C = ROLT^{30}(B), A = T$$

1.1.6 密码算法之SHA2

1) 算法概述

SHA2算法有消息预处理和哈希函数的主循环两个过程，下述为两个步骤的过程：

①消息预处理

在计算消息的哈希值之前，需要对消息进行预处理，主要是补位和消息分块。

补位：假设消息M的二进制编码长度为l位，首先在消息末尾补上一位“1”，然后再补上k个“0”，其中k为下列方程的最小非负整数：

$$l + 1 + k \equiv 448 \pmod{512}$$

然后，再在上述字符串后面补上l的二进制表示形式。最终补完以后，消息二进制位数长度是512的倍数。

消息分块：将补码处理后的消息以512位为单位分块为： $M^{(1)}$ 、 $M^{(2)}$ 、... $M^{(N)}$ 。其中第i个消息块的前32位表示为： $M_0^{(i)}$ ，第二个32位为： $M_1^{(i)}$ ，以此类推，最后32位的消息块可表示为：

$M_{15}^{(i)}$ 。

1.1.6 密码算法之SHA2

1) 算法概述

② 哈希函数的主循环

SHA-2算法的压缩函数逻辑如下：

For $i = 1 \rightarrow N$ ($N =$ 补码后消息块个数)

用第 $(i - 1)$ 次哈希值来对 a, b, c, d, e, f, g, h 进行初始化，当 $i = 1$ 时，使用初始化哈希值 $H^((0))$ ，即：

$$a \leftarrow H_1^{(i-1)}$$

$$b \leftarrow H_2^{(i-1)}$$

...

$$h \leftarrow H_8^{(i-1)}$$

1.1.6 密码算法之SHA2

1) 算法概述

应用SHA-2的压缩函数循环64次来更新a,b,...,h。即：

$$T_1 \leftarrow h + \Sigma_1(e) + Ch(e, f, g) + K_j + W_j$$

$$T_2 \leftarrow \Sigma_0(a) + M_{aj}(a, b, c)$$

$$h \leftarrow g$$

$$g \leftarrow f$$

$$f \leftarrow e$$

$$e \leftarrow d + T_1$$

$$d \leftarrow c$$

$$c \leftarrow b$$

$$b \leftarrow a$$

$$a \leftarrow T_1 + T_2$$

1.1.7 密码算法之SM2

1) 算法概述

SM2算法作为公钥算法，可以完成签名以及加解密应用等。因此SM2算法标准确定了标准过程有签名验签、加密解密：

(1) 签名过程

SM2数字签名算法适用于商用密码应用中的数字签名和验证。发送方可以使用私钥对数据进行签名，接收方使用相应的公钥来验证签名的有效性。

假设用户A作为签名者，拥有长度为 len_A 比特的可辨别标识 ID_A ，私钥 d_A ，公钥 $P_A = (x_A, y_A)$ ，待签名的消息为M。

①计算 $M^* = Z_A \parallel M$ 。输入标识 ID_A 和公钥产生一个Z值，然后Z值和M进行拼接产生 M^* 。

②计算 $e = H_v(M^*)$ ，将产生的 M^* 进行Hash计算。按规则将e的数据类型转换为整数。

③产生随机数 $k \in [1, n - 1]$ 。此处的n为奇素数。

④计算椭圆曲线点 $(x_1, y_1) = [k]G$ ，按规则将 x_1 的数据类型转换为整数。

($[k]G$ 表示 $k \cdot G$)

⑤计算 $r = (e + x_1) \bmod n$ ，若 $r = 0$ 或 $r + k = n$ 则返回第3步。

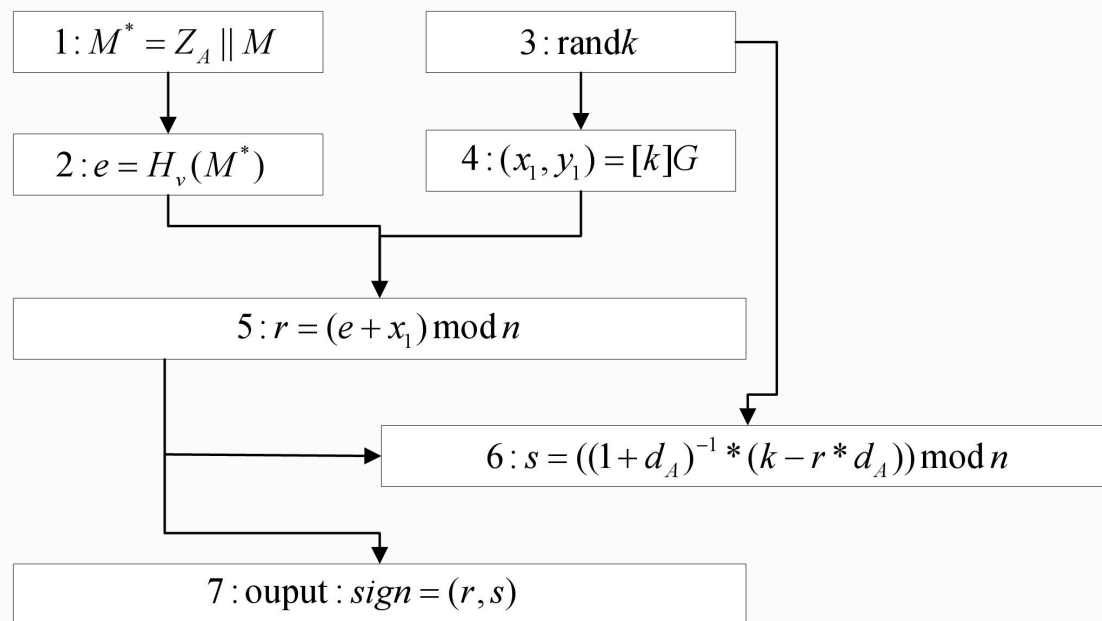
⑥计算 $s = ((1 + d_A)^{-1} * (k - r * d_A)) \bmod n$ ，若 $s = 0$ 则返回第3步。

⑦按规则将r、s的数据类型转换为字符串，得到消息M的签名为(r, s)。

1.1.7 密码算法之SM2

1) 算法概述

流程图：



1.1.7 密码算法之SM2

1) 算法概述

加密算法流程： 设消息为比特串 M ， $klen$ 为 M 的比特长度， P_B 表示接收者公钥。

①用随机数发生器产生随机数 $K \in [1, n - 1]$;

② 计算椭圆曲线点 $C_1 = [k]G = (x_1, y_1)$, ($[k]G$ 表示 $[k]*G$, G 表示椭圆曲线上的一个点)。

③计算椭圆曲线点 $S = [h] * P_B$, 若 S 是无穷远点, 则返回步骤1。

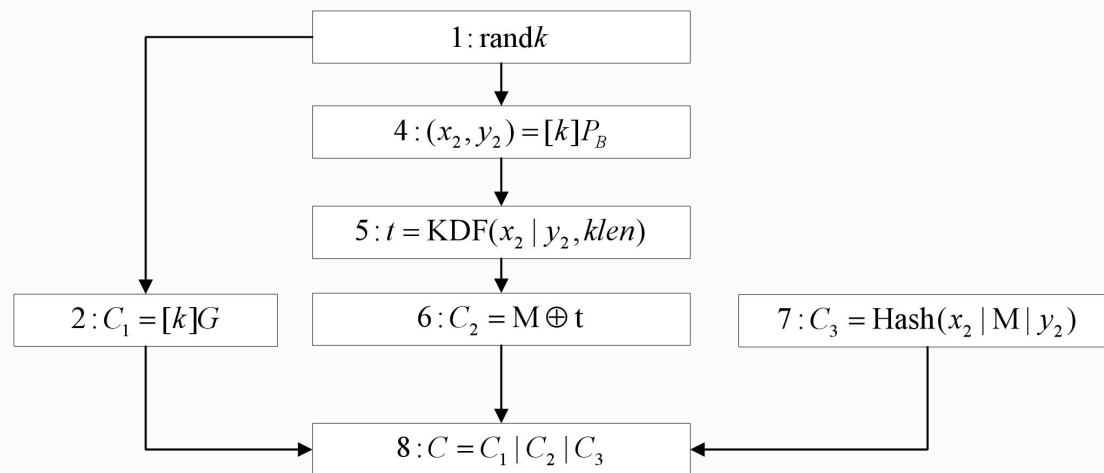
④ 计算椭圆曲线点 $[k] * P_B = (x_2, y_2)$ 。

⑤ 计算 $t = \text{KDF}(x_2 | y_2, klen)$ 。
(KDF 表示密钥派生函数)。若 t 为全0比特串, 则返回步骤1。

⑥计算 $C_2 = M \oplus t$ 。

⑦计算 $C_3 = \text{Hash}(x_2 | M | y_2)$ 。

⑧输出密文 $C = C_1 | C_2 | C_3$ 。



1.1.8 密码算法之SM3

1) 算法概述

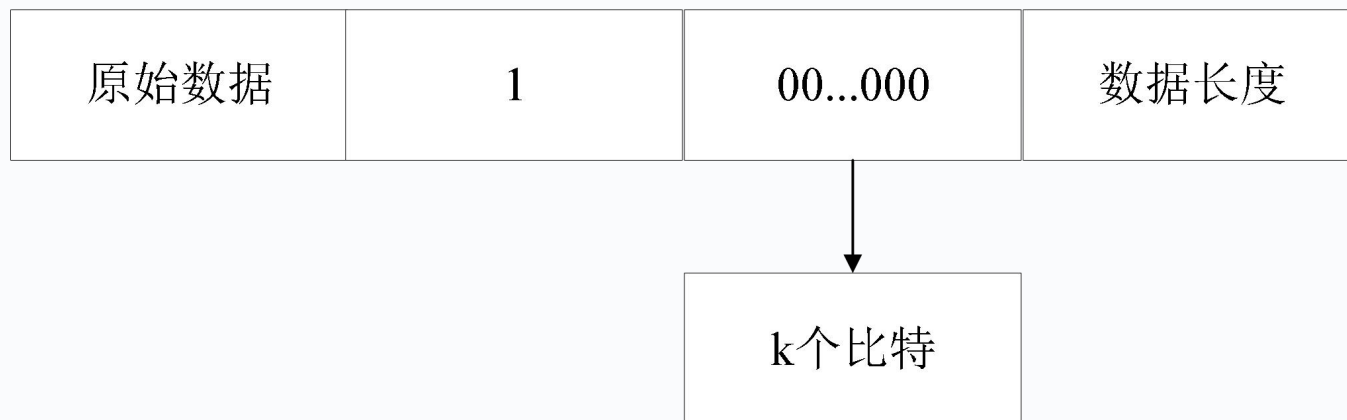
消息填充

SM3的消息扩展步骤是以512位的数据分组作为输入的。因此，我们需要在一开始就把数据长度填充至512位的倍数。具体步骤如下：

①先填充一个“1”，后面加上k个“0”。其中k是满足 $(n+1+k) \bmod 512 = 448$ 的最小正整数。

②追加64位的数据长度。

填充完的数据大概如下：



1.1.8 密码算法之SM3

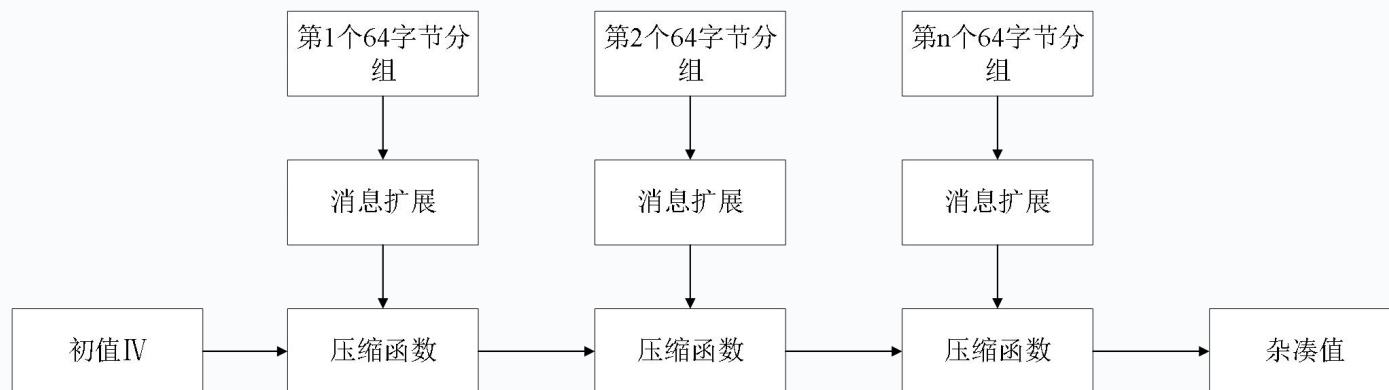
1) 算法概述

消息扩展

SM3的迭代压缩步骤没有直接使用数据分组进行运算，而是使用这个步骤产生的132个消息字(一个消息字的长度为32位)。概括来说，先将一个512位数据分组划分为16个消息字，并且作为生成的132个消息字的前16个。再这16个消息字递推生成剩余的116个消息字。在最终得到的132个消息字中，前68个消息字构成数列 $\{W_j\}$ ，后64个消息字构成数列 $\{W_j^*\}$ ，其中下标 j 从0开始计数。

迭代压缩

对每一个分组都进行消息扩展，在借助寄存器得到最后的杂凑值（哈希值）。迭代压缩这一步有很多的中间变量和寄存器的操作。这个迭代过程如下图所示：

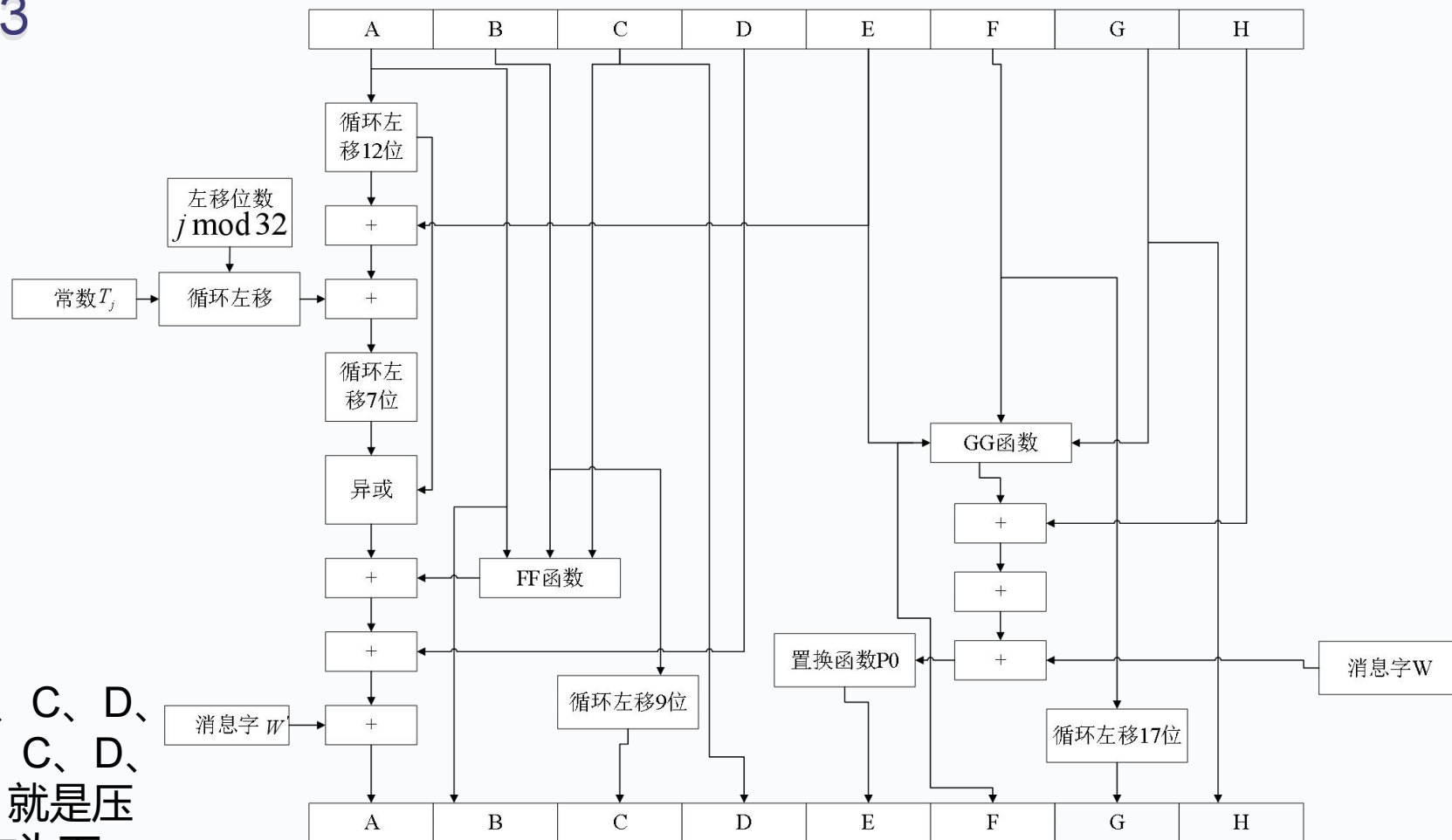


1.1.8 密码算法之SM3

1) 算法概述

初始值IV被放在A、B、C、D、E、F、G、H八个32位变量中，整个算法中最核心、最复杂的部分就在于压缩函数。压缩函数将这八个变量进行64轮相同的计算，每一轮的计算过程如右图所示：

最后，再将计算完成的A、B、C、D、E、F、G、H和原来的A、B、C、D、E、F、G、H分别进行异或，就是压缩函数的输出。这个输出再作为下一次调用压缩函数时的初值。依次类推，指导用完最后一组132个消息字为止。



1.1.9 密码算法之SM4

1) 算法概述

合成置换T:

①非线性变换 τ

τ 由4个并行的S盒构成。设输入为 $A=(a_0, a_1, a_2, a_3) \in (Z_2^8)^4$, 输出为 $B=(b_0, b_1, b_2, b_3) \in (Z_2^8)^4$, 则 $(b_0, b_1, b_2, b_3) = \tau(A) = (\text{Sbox}(a_0), \text{Sbox}(a_1), \text{Sbox}(a_2), \text{Sbox}(a_3))$ 。

②线性变换L

非线性变换 τ 的输出时线性变换L的输入。设输入为 $B \in Z_2^{32}$, 输出为 $C \in Z_2^{32}$, 则 $C = L(B) =$

$B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24)$ 。S盒使用规则: 设S盒的输入为EF, 则经S盒运算的输出结果为表中的第E行、第F列中的值, 即 $\text{Sbox}(EF) = 0x84$ 。S盒如下图所示:

1.1.9 密码算法之SM4

1) 算法概述

$\begin{smallmatrix} Y \\ \diagdown \\ X \end{smallmatrix}$	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	D6	90	E9	FE	CC	E1	3D	B7	16	B6	14	C2	28	FB	2C	05
1	2B	67	9A	76	2A	BE	04	C3	AA	44	13	26	49	86	06	99
2	9C	42	50	F4	91	EF	98	7A	33	54	0B	43	ED	CF	AC	62
3	E4	B3	1C	A9	C9	08	E8	95	80	DF	94	FA	75	8F	3F	A6
4	47	07	A7	FC	F3	73	17	BA	83	59	3C	19	E6	85	4F	A8
5	68	6B	81	B2	71	64	DA	8B	F8	EB	0F	4B	70	56	9D	35
6	1E	24	0E	5E	63	58	D1	A2	25	22	7C	3B	01	21	78	87
7	D4	00	46	57	9F	D3	27	52	4C	36	02	E7	A0	C4	C8	9E
8	EA	BF	8A	D2	40	C7	38	B5	A3	F7	F2	CE	F9	61	15	A1
9	E0	AE	5D	A4	9B	34	1A	55	AD	93	32	30	F5	8C	B1	E3
A	1D	F6	E2	2E	82	66	CA	60	C0	29	23	AB	0D	53	4E	6F
B	D5	DB	37	45	DE	FD	8E	2F	03	FF	6A	72	6D	6C	5B	51
C	8D	1B	AF	92	BB	DD	BC	7F	11	D9	5C	41	1F	10	5A	D8
D	0A	C1	31	88	A5	CD	7B	BD	2D	74	D0	12	B8	E5	B4	B0
E	89	69	97	4A	0C	96	77	7E	65	B9	F1	09	C5	6E	C6	84
F	18	F0	7D	EC	3A	DC	4D	20	79	EE	5F	3E	D7	CB	39	48

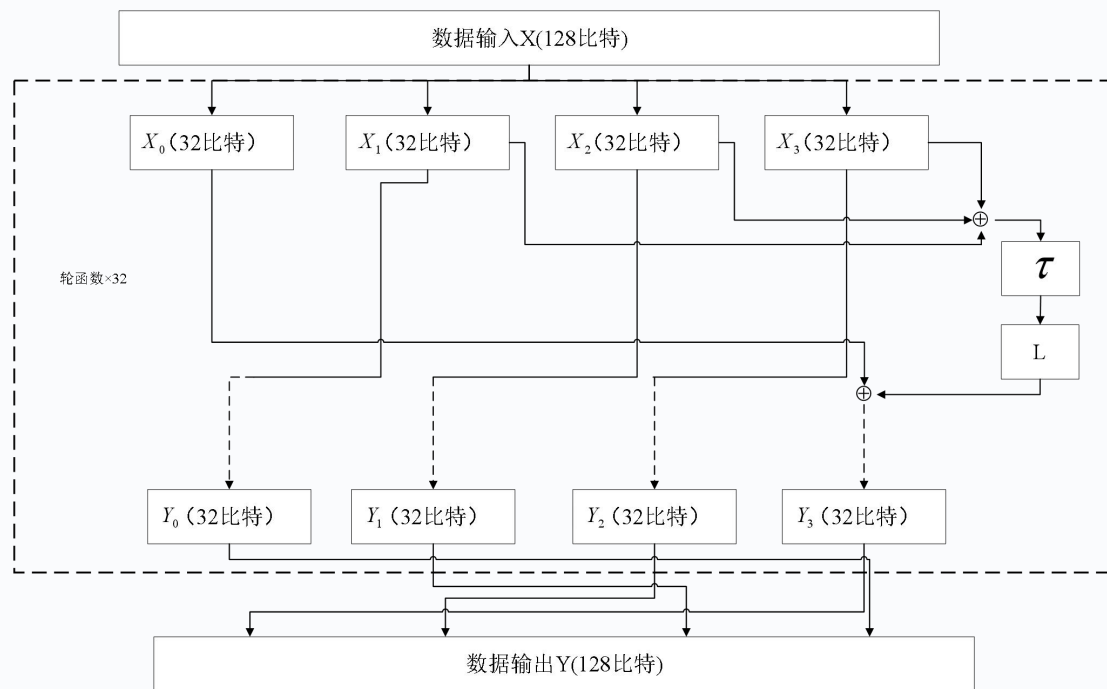
合成置换 $T: Z_2^{32} \rightarrow Z_2^{32}$, 是一个可逆变换, 由非线性变换 τ 和线性变换 L 复合而成, 即 $T(.) = L(\tau(.))$ 。

1.1.9 密码算法之SM4

1) 算法概述

轮函数F

本算法采用非线性迭代结构，以字为单位进行加密运算，称一次迭代运算为一轮变换。设输入为 $(X_0, X_1, X_2, X_3) \in (Z_2^{32})^4$ ，轮密钥为 $rk \in Z_2^{32}$ ，则轮函数F为： $F(X_0, X_1, X_2, X_3, rk) = X_0 \oplus T(X_1 \oplus X_2 \oplus X_3 \oplus rk)$ 。示意图如下：





谢谢！